

第 18 章: Arguments (参数传递)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 第 17 章研究了作用域 (scopes) ——变量在哪里定义、在哪里查找; 本章研究函数故事的另外半壁江山: 参数传递 (argument passing) ——对象如何被作为输入送进函数。你将看到, 参数本质上就是"按赋值传递" (即传对象引用, call by sharing), 并在此基础上掌握关键字 (keyword) 参数、默认值 (default)、args/kwarg s 收集与解包、仅关键字参数 (keyword-only)、仅位置参数 (positional-only) 等一整套匹配工具。这是读懂 Python 标准库与第三方框架调用签名的关键一章, 也为第 19 章的函数注解、递归与函数式工具打下基础。

18.1 开篇引言 (Chapter Opener)

英文原文

The preceding chapter explored Python's scopes—the places where variables are defined and looked up. As we saw, the place where a name is defined in our code determines much of its meaning. This chapter continues the function story by studying the concepts in Python argument passing—the way that objects are sent to functions as inputs. As you'll see, argument

s (a.k.a. parameters) are assigned to names in a function, but have more to do with object references than with variable scopes. You'll also find that Python provides extra tools, such as keywords, defaults, and argument collectors and extractors, that allow arguments to be sent to functions flexibly.

中文翻译

上一章探索了 Python 的作用域 (scopes) ——变量被定义和查找的地方。如我们所见，名字 (name) 在代码中被定义的位置决定了它的大部分含义。本章延续函数的话题，研究 Python 参数传递 (argument passing) 中的概念——对象作为输入被发送给函数的方式。你会看到，参数 (argument, 又称形参 parameter) 在函数中被赋值给名字，但它们更多与对象引用 (object reference) 有关，而不是与变量作用域有关。你还会发现，Python 提供了额外的工具，比如关键字 (keywords)、默认值 (defaults)、参数收集器和提取器 (argument collectors and extractors)，让参数可以被灵活地发送给函数。

深度理解

·核心概念：本章是"对象如何进入函数"的完整说明书。核心只有一句话：参数传递 = 把对象引用赋值给函数的局部名字。

·与上一章的关系：第 17 章讲"名字在哪里" (作用域)，本章讲"对象怎么进门" (参数)。两者交汇于一个事实：函数头里的参数名，只是函数局部作用域里的普通名字。

·底层视角：参数传递背后没有任何魔法，就是自动化的赋值语句（assignment）。所有在赋值中学到的规则——不复制对象、共享引用、原地修改穿透影响——在这里原样生效。

·为什么这样设计：统一性（uniformity）。Python 把"传参"降维成"赋值"，使语言只有一种数据流机制，降低学习成本。

·初学者误区：很多人以为"参数是复制一份值传进去"。真相是：一个字节的數據都不会被复制，传进去的是指向对象的指针。

18.2 参数传递基础 (Argument-Passing Basics)

英文原文

Earlier in this part of the book, we learned that `def` and `lambda` are function definitions, and both include an argument-list headers that name variables which receive values passed by calls. These arguments are used in function bodies, and may be matched between call and header by position, name, and other means we'll explore later in this chapter. More fundamentally, though, it was also noted that all Python arguments are passed by assignment—which means object reference. This has some subtle ramifications that aren't always obvious to newcomers. Let's start our arguments a

dventure by exploring how this works. Here is a rundown of the key points in this model: Arguments are passed by automatically assigning objects to local variable names. Function arguments are just another instance of Python assignment at work: they are references to objects sent by, and possibly shared with, the caller. Because references are implemented as pointers, all arguments are passed by opaque pointer. As for all assignments, objects passed as arguments are never automatically copied. Assigning to argument names inside a function does not affect the caller. Per assignment norms, when the function is run by a call, argument names in the function header simply become new names in the local scope of the function. There is no aliasing between function argument names and variable names in the caller's scope. Changing a mutable object argument in a function may impact the caller. On the other hand, as arguments are simply references to passed-in objects, functions can change passed-in mutable objects in place, and the results may affect the caller. Hence, mutable arguments can be both input and output for functions. For more details on references, see Chapter 6; everything we studied there also applies to function arguments, though the assignment to argument names is automatic and implicit.

中文翻译

在本部分前面我们学过，`def` 和 `lambda` 都是函数定义 (function definitions)，两者都包含参数列表头 (argument-list header)，其中命名了一些变量，用来接收调用 (calls) 传来的值。这些参数在函数体中使用，并可以通过位置 (position)、名字 (name) 以及本章后面将要探索的其他方式，在调用与函数头之间进行匹配。但更根本的是，之前也指出过：Python 的所有参数都是按赋值传递的——也就是传对象引用。这带来了一些细微的推论，新手往往看不透。让我们从探索这一点开始参数之旅。下面是这个模型关键点的概要：参数通过“自动把对象赋值给局部变量名”来传递。函数参数只是 Python 赋值在工作的又一个实例：它们是调用方 (caller) 发来的、可能与调用方共享的对象的引用 (references)。因为引用是用指针 (pointers) 实现的，所有参数都以不透明的指针传递。和所有赋值一样，作为参数传入的对象永远不会被自动复制。在函数内部给参数名赋值不会影响调用方。按照赋值的惯例，当函数被调用执行时，函数头中的参数名只是在函数的局部 (local) 作用域中成为新名字。函数参数名与调用方作用域中的变量名之间不存在别名 (aliasing) 关系。在函数中修改可变对象参数可能会影响调用方。另一方面，由于参数只是对传入对象的引用，函数可以原地 (in place) 修改传入的可变对象，其结果可能影响到调用方。因此，可变 (mutable) 参数既可以作为函数的输入，也可以作为函数的输出。关于引用的更多细节，参见第 6 章；我们在那里研究的所有内容同样适用于函数参数，只不过对参数名的赋值是自动、隐式的。

深度理解

·核心概念：参数传递三定律——① 传引用、不复制；② 给参数名重新赋值，只改变函数局部名字，不影响调用方；③ 对可变对象做原地修改，会穿透函数边界、影响调用方。这是本章的地基，后面所有内容都是在这三定律上叠加匹配语法。

·底层实现：CPython 中，每个对象是堆上的一个结构体，变量名只是指向对象的指针。调用函数时，解释器先把实参对象的指针压入求值栈，再创建新的栈帧（frame），按匹配规则把指针填入新帧的“快速局部变量槽”（fast locals）。整个过程没有任何对象数据被复制。

·“按共享传递”（call by sharing）：语义学家对这种混合模型的标准称呼——传的是共享引用：不可变对象表现像“按值传递”（因为改不了），可变对象表现像“按指针传递”（因为能原地改）。Java、Ruby、JavaScript 都是这个模型。

·实际问题：正因传引用不复制，把大列表、大字典传给函数几乎零成本；也正是因为原地修改会外溢，函数可以就地更新容器，充当“输出参数”。

·初学者误区：① 以为 `a = 99` 会“改掉”外面的变量——不会，那只是让局部名字改指另一个对象；② 以为传列表就是“复制一份”——不，外面和里面指向同一个对象。

·生活类比：把房间钥匙（引用）递给管理员。管理员把钥匙挂到另一扇门上（重绑定名字）——你的门没变；但管理员进屋重新装修（原地修改对象）——你的房间就真的

变了。

18.3 参数与共享引用 (Arguments and Shared References)

英文原文

To demo argument-passing properties at work, consider the following code:

```
python >>> def f(a): # a is assigned a reference to the
passed object ... a = 99 # Changes local variable a only
>>> b = 88 >>> f(b) # a and b both reference same 88
initially >>> b # But b is not changed by assignment
to a in f 88
```

In this example, the variable `a` is assigned the object 88 at the moment the function is called with `f(b)`, but `a` lives only within the called function's local scope. Changing `a` inside the function has no effect on the place where the function is called; it simply resets the local variable `a` to a completely different object, 99. That's what is meant by a lack of name aliasing—assignment to an argument name inside a function (e.g., `a = 99`) does not magically change a variable like `b` in the scope of the function call. Argument names may share passed objects initially (they are essentially pointers to those objects), but only temporarily, when the function is first called. As soon as an argument name

is reassigned, this relationship ends. At least, that's the case for assignment to argument names themselves. When arguments are passed mutable objects like lists and dictionaries, we also need to be aware that in-place changes to such objects may live on after a function exits, and hence impact callers. Here's an example that demonstrates this behavior:

```
python >>> def changer(a, b): # Arguments assigned
references to objects ... a = 2 # Changes local name's
value only ... b[0] = 'mod' # Changes shared object in
place: outlives call >>> X = 1 >>> L = [1, 2] # Caller:
>>> changer(X, L) # Pass immutable and mutable ob
jects >>> X, L # X is unchanged, but L is different! (1,
['mod', 2])
```

In this code, the `changer` function assigns values to a argument `a` itself, and to a component of the object referenced by argument `b`. These two assignments within the function are only slightly different in syntax but have radically different results: Because `a` is a local variable name in the function's scope, the first assignment has no effect on the caller—it simply changes the local variable `a` to reference a completely different object, and does not change the value of the name `X` in the caller's scope. This is the same as in the prior example. Argument `b` is a local variable name, too, but it is passed a mutable object—the list that `L` also references in the caller's scope. Because the assignment to `b[0]` in the function is an in-place change to a share

d object, its result impacts the value of L after the function returns. Really, the second assignment statement in `changer` doesn't change `b`—it changes part of the object that `b` currently references. This in-place change impacts the caller only because the changed object outlives the function call. The name `L` hasn't changed either—it still references the same, changed object—but it seems as though `L` differs after the call because the value it references has been modified within the function. In effect, the list name `L` serves as both input to and output from the function. Figure 18-1 illustrates the name/object bindings that exist immediately after the function has been called, and before its code has run. When the call begins, two objects are shared among four names. If this example is still confusing, it may help to notice that the effect of the automatic assignments of the passed-in arguments is the same as running a series of simple assignment statements. In terms of the first argument, the assignment has no effect on the caller:

```
python >>> X = 1 >>> a = X # They share the same object
>>> a = 2 # Name change resets 'a' only, 'X' is still 1
>>> X
1
```

Figure 18-1. Function arguments and shared object references The assignment through the second argument does affect a variable at the call, though, because it is an in-place object change:

```
python >>> L = [1, 2] >>> b = L # They share the same object >>> b[0] = 'mod' # In-place change from 'b' to 'L' sees the change too >>> L ['mod', 2]
```

If you recall our discussions about shared mutable objects in Chapters 6 and 9, you'll recognize the phenomenon at work: changing a mutable object in place can impact other references to that object. Here, the effect is to make one of the arguments work like both an input and an output of the function.

中文翻译

为了演示参数传递特性在实战中的表现，考虑如下代码：

```
python >>> def f(a): # a 被赋值为传入对象的引用... a = 99 # 只改变局部变量 a >>> b = 88 >>> f(b) # 开始时 a 和 b 都引用同一个 88 >>> b # 但 f 里对 a 的赋值不会改变 b 88
```

在这个例子中，当用 `f(b)` 调用函数时，变量 `a` 被赋值为对象 `88`，但 `a` 只存在于被调用函数的局部作用域内。在函数内改变 `a` 对调用点没有任何影响——它只是把局部变量 `a` 重置为另一个完全不同的对象 `99`。这就是“没有名字别名”的含义——在函数内对参数名的赋值（如 `a=99`）不会神奇地改变调用作用域中像 `b` 这样的变量。参数名最初可能共享传入的对象（它们本质上是指向这些对象的指针），但这只是暂时的——仅在函数第一次被调用时发生。一旦参数名被重新赋值，这种关系就结束了。至少，对于“给参数名本身赋值”是这种情况。当参数传入的是列表、字典这样的可变对象时，我们还需要意识到：对这种对象的原地修改会在函数退出后依然存在，从而影响调用方。下面是一个演示这种行为的例子：

```
python >>> def changer(a,
```

b): # 参数被赋值为对象的引用... a = 2 # 只改变局部名字的值... b[0] = 'mod' # 原地修改共享对象：效果比调用存续更久>>> X = 1 >>> L = [1, 2] # 调用方：>>> changer(X, L) # 传入不可变对象和可变对象>>> X, L # X 没变，但 L 不一样了！(1, ['mod', 2]) 在这段代码中，changer 函数给参数 a 本身赋值，也给参数 b 所引用对象的一个组件赋值。函数内这两个赋值语句在语法上只有细微差别，但结果截然不同：由于 a 是函数作用域中的局部变量名，第一个赋值对调用方没有任何影响——它只是把局部变量 a 改为引用一个完全不同的对象，并没有改变调用方作用域中名字 X 的值。这与上一个例子相同。参数 b 也是一个局部变量名，但它传入的是一个可变对象——调用方作用域中 L 也引用的那个列表。因为函数中对 b[0] 的赋值是对共享对象的原地修改，其结果在函数返回后影响了 L 的值。实际上，changer 中的第二条赋值语句并没有改变 b——它改变的是 b 当前所引用的对象的一部分。这种原地修改之所以影响调用方，仅仅是因为被修改的对象比函数调用活得更久。名字 L 也没有变——它仍然引用同一个（被修改过的）对象——但调用之后 L 看起来确实变了，因为它引用的值在函数内部被修改了。实际上，列表名 L 同时充当了函数的输入和输出。图 18-1 展示了函数被调用之后、其代码运行之前那一瞬间的名字/对象绑定关系。调用开始时，两个对象被四个名字共享。如果这个例子仍让你困惑，有个办法可能有帮助：注意传入参数时的自动赋值，其效果与执行一连串普通赋值语句完全一样。就第一个参数而言，赋值对调用方没有影响：python >>> X = 1 >>> a = X # 它们共享同一个对象>>> a = 2 # 改变名字只重置了 a，X 仍是 1 >>> X 1 图 18-1. 函数参数与共享的对象引用但通过第二个

参数的赋值确实会影响调用点的变量，因为它是对象的原地修改：`python >>> L = [1, 2] >>> b = L # 它们共享同一个对象 >>> b[0] = 'mod' # 从 b 做的原地修改：L 也看到这个变化 >>> L ['mod', 2]` 如果你还记得第 6 章和第 9 章关于共享可变对象的讨论，你会认出这里的现象：原地修改一个可变对象，会影响该对象的其他引用。在这里，其效果就是让一个参数既像函数的输入、又像函数的输出。

深度理解

·核心概念：调用函数的那一刻，只发生“名字绑定”，不发生数据复制。`f(b)` 时局部名字 `a` 与调用方名字 `b` 暂时指向同一个对象；`a = 99` 之后绑定关系即断裂。

·底层实现：`b[0] = 'mod'` 在字节码层面是 `STORESUBSCR`——它拿到列表对象的指针，直接写入列表内部存放元素指针的数组。写的是对象本身，所以不管谁引用这个对象，看到的都是修改后的内容。而 `a = 2` 只是 `STOREFAST`，改的是栈帧里 `a` 这个槽位的指针。两条指令作用的目标完全不同。

·为什么这样设计：统一“变量 = 名字指向对象”的心智模型。Python 里不存在“改变变量的值”这种操作，只存在“让名字指向另一个对象”或“修改对象本身”。

·实际问题：这就是“函数返回后我的列表怎么被改了”这类 bug 的根源；也是“就地排序 vs 返回新列表”这类 API 设计决策的依据。

·初学者误区：把变量当成“盒子”，总想“把盒子里的值换掉”。正确的模型是把名字当“贴在对象上的标签”——重贴

标签 vs 修改对象，是两回事。

·生活类比：图 18-1 展示的"两个对象被四个名字共享"就像：一把钥匙配了两把（X 和 a 指向 88，L 和 b 指向同一个列表）。一个人换挂钥匙不影响别人；但一个人把屋里的墙刷了，所有拿钥匙的人进屋都会看到新墙。

代码分析

```
>>> def f(a):           # a
...     a = 99          # a
>>> b = 88
>>> f(b)                # a b 88
>>> b                   # f a b
88
```

解释器如何处理：

1. `def f(a):` 执行时，解释器创建一个函数对象并保存参数名 `a`；此刻不执行函数体。
2. `b = 88`：88 是小整数缓存（small int cache，CPython 预缓存 -5~256）里的对象，`b` 指向它。
3. `f(b)`：解释器把 `b` 指向的对象（整数 88）压栈，创建新栈帧，把 `a` 槽位指向 88。此时 `a` 和 `b` 共享对象。
4. 函数体内 `a = 99`：STOREFAST 把 `a` 槽位改为指向另一个整数对象 99。88 的引用计数从 2 降为 1。
5. 函数返回，帧销毁，`a` 不复存在；`b` 仍指向 88，所以打印 88。

```
>>> def changer(a, b):    #
...     a = 2            #
...     b[0] = 'mod'     #
>>> X = 1
>>> L = [1, 2]
>>> changer(X, L)
>>> X, L
(1, ['mod', 2])
```

解释器如何处理：

- `a = 2`: `STOREFAST`，只改帧中槽位。X 不受影响（整数 1 也不可变，没有“原地改”这回事）。
- `b[0] = 'mod'`: `STORESUBSCR`。解释器找到 `b` 指向的列表对象，把它的第 0 个槽位从指向整数 1 改为指向字符串 'mod'。列表对象本身的内存没换——L 依然指向它——但它内部变了。
- 函数返回后，L 指向的仍是同一个列表，所以 X, L 显示 (1, ['mod', 2])。

设计思想：用“名字绑定 + 对象可变性”两要素的组合，Python 只靠赋值机制就同时实现了“按值”和“按引用”两种其他语言需要两套语法才能表达的行为。

18.4 避免可变参数被修改 (Avoiding Mutable Argument Changes)

英文原文

This behavior of in-place changes to mutable arguments isn't a bug—it's simply the way argument passing

works in Python, and turns out to be widely useful in practice. Arguments are normally passed to functions by reference because that is what we normally want. It means we can pass large objects around our programs without making multiple copies along the way, and we can easily update these objects as we go. In fact, as you'll see in Part VI, Python's class and OOP model depends upon changing a passed-in "self" argument in place, to update mutable object state. If we don't want in-place changes within functions to impact objects we pass to them, though, we can simply make explicit copies of mutable objects, as we saw in Chapter 6. For function arguments, we can always copy the list at the point of call with tools like `list`, `list.copy`, or an empty slice (dictionaries have similar copy tools):

```
python L = [1, 2] changer(X, L[:]) # Pass a copy, so our 'L' does not change
```

We can also copy within the function itself, if we never want to change passed-in objects, regardless of how the function is called:

```
python def changer(a, b): b = b.copy() # Copy input list so we don't impact caller a = 2 b[0] = 'mod' # Changes our list copy only
```

Both of these copying schemes don't stop the function from changing the object—they just prevent those changes from impacting the caller. To really preven

t changes, we can always convert to immutable objects to force the issue. Tuples, for example, raise an exception when changes are attempted:

```
python L = [1, 2] changer(X, tuple(L)) # Pass a tuple, so changes are errors
TypeError: 'tuple' object does not support item assignment
```

This scheme uses the built-in tuple function, which builds a new tuple out of all the items in a sequence (really, any iterable). It's also something of an extreme—because it forces the function to be written to never change passed-in arguments, this solution might impose more limitations on the function than it should, and so should generally be avoided (you never know when changing arguments might come in handy for other calls in the future). Using this technique will also make the function lose the ability to call any list-specific methods on the argument, including methods that do not change the object in place (e.g. copy, though tuples have adopted list count and index). The main point to remember here is that functions might update mutable objects like lists and dictionaries passed into them. This isn't necessarily a problem if it's expected, and often serves useful purposes. Moreover, functions that change passed-in mutable objects in place are probably designed and intended to do so—the change is likely part of a well-defined API that you shouldn't violate by making copies. However, you do have to be aware of this property—if objects change

out from under you unexpectedly, check whether a called function might be responsible, and make copies when objects are passed if needed.

中文翻译

对可变参数做原地修改这种行为不是 bug——它只是 Python 参数传递的工作方式，而且在实践中被证明广泛有用。参数通常按引用传给函数，因为那正是我们想要的：我们可以在程序中传递大对象，而不必沿途复制多份，并且可以轻松地在边传边更新这些对象。事实上，正如你将在第六部分看到的，Python 的类与 OOP 模型依赖于原地修改传入的 "self" 参数，来更新可变对象的状态。不过，如果我们不希望函数内的原地修改影响到传入的对象，可以像第 6 章那样显式复制可变对象。对于函数参数，我们可以在调用点用 `list`、`list.copy` 或空切片等工具复制列表（字典有类似的复制工具）：`python L = [1, 2] changer(X, L[:])` # 传一个副本，这样我们的 L 不会被改变如果我们无论函数如何被调用，都不想修改传入的对象，也可以在函数内部复制：`python def changer(a, b): b = b.copy() # 复制输入列表，这样不会影响调用方``a = 2 b[0] = 'mod'` # 只修改我们的列表副本这两种复制方案并不能阻止函数修改对象——它们只是防止这些修改影响调用方。要真正阻止修改，我们可以把对象转换为不可变对象来强制实现。例如，元组 (tuple) 在被尝试修改时会抛出异常：`python L = [1, 2] changer(X, tuple(L))` # 传入元组，这样修改就会报错 `TypeError: 'tuple' object does not support item assignment` 这个方案使用内置 `tuple` 函数，它从一个序列（实际上可以是任何可迭代对象）的所有元素构建一个新元组。这多少有点极端——

—因为它强迫函数被写成"永远不修改传入的参数", 这个方案可能会给函数强加超出必要的限制, 因此一般应该避免 (你永远不知道未来其他调用中, 修改参数会不会派上用场)。使用这个技巧还会使函数失去对参数调用任何列表专属方法的能力, 包括那些并不原地修改对象的方法 (如 `copy`, 尽管元组已经采纳了列表的 `count` 和 `index`)。这里要记住的要点是: 函数可能会更新传入的可变对象 (如列表和字典)。如果这是可预期的, 就不一定是问题, 而且常常有实用价值。此外, 那些原地修改传入可变对象的函数, 很可能是特意设计成这样的——这种修改往往是一个定义良好的 API 的一部分, 你不应该通过复制副本来破坏它。然而, 你必须意识到这一特性——如果对象在你眼皮底下意外地变了, 检查一下是否是某个被调用的函数干的, 并在必要时在传对象时做副本。

深度理解

- 核心概念: 防御策略的三种梯度——调用方复制、函数内复制、转成不可变对象。前两种只是"隔离", 第三种才是"禁止"。
- 底层实现: `L[:]` (空切片) 在 CPython 中调用列表的切片协议, 分配新列表并逐个拷贝元素指针 (浅拷贝, `shallow copy`); `list.copy` 是同构操作。`tuple(L)` 则新建一个元组对象——元组对象内部缓冲区不可写, `STORESUBSCR` 会触发 `TypeError`。
- 为什么这样设计: 默认"不复制"符合性能直觉; 把"要不要隔离"的决定权交给调用方/函数作者——这正是"显式优于隐式" (`explicit is better than implicit`) 的体现。

·实际问题：OOP 中 self 的原地修改是常态——list.append 本身就是"修改传入对象"的范例。API 文档里写明"本函数会就地修改实参"是一种专业契约。

·初学者误区：① 以为传 tuple(L) 能"一劳永逸防修改"——实际上它限制了函数对参数的合法操作（连 list.copy 都调不了），得不偿失；② 分不清"防御性复制"和"防止修改"：复制只是不让修改波及调用方。

·生活类比：发文件给别人——不想让他在原件上乱改，可以给他复印件（复制）；想彻底不让他改，就给他只读的 PDF（元组）——但 PDF 也意味着他没法帮你做任何编辑了。

18.5 模拟输出参数与多结果返回 (Simulating Output Parameters and Multiple Results)

英文原文

Here's another function topic from the assignments department. We've already discussed the return statement and used it in examples. What we haven't yet seen is a common though unusual coding technique it enables: because return can send back any sort of object, it can return multiple values by packaging them in a tuple or other collection type. In fact, although Python doesn't support what some languages label "call by reference" argument passing, we can simulate it

t by returning tuples and assigning the results back to the original argument names in the caller: python

```
>>> def multiple(x, y): ... x = 2 # Changes local names only ... y = [3, 4] ... return x, y # Return multiple new values in a tuple
>>> X = 1
>>> L = [1, 2]
>>> X, L = multiple(X, L) # Assign results to caller's names
>>> X, L
(2, [3, 4])
```

It looks like the code is returning two values here, but it's really just one—a two-item tuple with the optional surrounding parentheses omitted. After the call returns, we can use tuple (a.k.a. sequence) assignment to unpack the parts of the returned tuple. (If you've forgotten why this works, flip back to "Tuples" in Chapters 4 and 9, and "Assignments" in Chapter 11.) The net effect of this coding pattern is to both send back multiple results and simulate the output parameters of other languages by explicit assignments. Here, X and L change after the call—but only because the code said so. Lists would work, too, but tuples sans () are less to type, and hence common.

中文翻译

这是“赋值部门”的又一个函数话题。我们已经讨论过 `return` 语句并在示例中使用过它。我们还没见过的，是它促成的一种常见但不太寻常的编码技巧：因为 `return` 可以送回任意种类的对象，它可以把多个值打包进一个元组或其他容器类型，从而“返回多个值”。事实上，虽然 Python 不支持某些语言所谓的“按引用调用”（call by reference）参数传递，但我们可以通过返回元组、再把结果赋值回调用方的原参

数名，来模拟它：`python >>> def multiple(x, y): ... x = 2 # 只改变局部名字... y = [3, 4] ... return x, y # 在元组中返回多个新值>>> X = 1 >>> L = [1, 2] >>> X, L = multiple(X, L) # 把结果赋值给调用方的名字>>> X, L (2, [3, 4])` 这里的代码看起来返回了两个值，但实际上只有一个——一个省略了可选外层括号的两元素元组。调用返回后，我们可以用元组（又称序列）赋值来解包返回元组的各个部分。（如果你忘了它为什么有效，翻回第 4 章和第 9 章的“元组”，以及第 11 章的“赋值”。）这种编码模式的净效果是：既送回多个结果，又通过显式赋值模拟了其他语言的输出参数（output parameters）。这里，`X` 和 `L` 在调用之后变了——但只是因为代码明确这样写了。列表也可以，但无括号的元组更少打字，因此更常见。

深度理解

·核心概念：Python 的“多返回值”是语法糖——真正返回的是一个元组对象；调用方用序列解包（sequence unpacking）把它拆开。

·底层实现：`return x, y` 编译为 `BUILDTUPLE`（把两个元素组成一个元组对象）再 `RETURNVALUE`；调用方 `X, L = ...` 编译为 `UNPACKSEQUENCE`（把元组弹出、按序赋值给两个目标槽位）。数据走的是“返回值通道”，与传参通道完全隔离。

·为什么这样设计：Lutz 点出关键——没有真正的“输出参数”，所有数据流向都经由返回值显式完成。这让“谁改了谁”一目了然，代码可预测性高。

- 生活类比：不是让管理员改你家的锁，而是管理员告诉你新锁的样式，你自己去买来换上（返回值 + 显式赋值）。
- 实际问题：`divmod()`、`dict.items()` 等都返回元组供解包；自定义函数返回 `(status, result)` 或使用 `dataclass/NamedTuple` 组织多结果，是这个模式的现代演进。
- 初学者误区：以为 `X, L = multiple(X, L)` 里的 `X`、`L` 与函数内的 `x`、`y` 有什么“暗线连接”——完全没有，它们只是名字相同而已，数据靠元组中转。

18.6 特殊的参数匹配模式 (Special Argument-Matching Modes)

英文原文

As we've just seen, arguments are always passed by assignment in Python; names in a `def` or `lambda` definition's header are assigned references to passed-in objects. On top of this model, though, Python provides additional tools that alter the way the argument objects in a call are matched with argument names in the definition prior to assignment. These tools are all optional, but allow us to code functions that support more flexible calling patterns and are commonly used by libraries you're likely to encounter. By default, arguments are matched between call and definition by position, from left to right, and you must pass exactly as many arguments as there are argument names in the

function definition. However, you can also specify matching by name, provide default values, unpack and collect arbitrarily many arguments, and even specify passing-mode requirements. This section presents all these extra tools with a quick overview followed by examples, and a formal look at how they interact at the end after we've had a chance to see the basics.

中文翻译

正如我们刚看到的，在 Python 中参数始终是按赋值传递的；`def` 或 `lambda` 定义头中的名字被赋值为传入对象的引用。在这个模型之上，Python 还提供了额外的工具，它们改变调用中的参数对象在赋值之前与定义中参数名的匹配（matched）方式。这些工具都是可选的，但能让我们编写支持更灵活调用模式的函数，也是你很可能遇到的库的常用手法。默认情况下，调用与定义之间的参数按位置匹配，从左到右，而且你必须传入与函数定义中参数名数量完全相同的参数。不过，你也可以按名字匹配、提供默认值、解包与收集任意数量的参数，甚至可以指定传递模式的要求。本节先快速概览所有这些额外工具，接着给示例，并在我们见过基础之后、于结尾对这些工具的相互作用做一次正式的审视。

深度理解

- 核心概念：匹配（matching）是“赋值之前”的分配阶段——先决定“哪个实参进哪个形参”，然后才执行赋值。底层的传递机制仍然是赋值。
- 为什么这样设计：这些工具主要是为库作者准备的，让库能提供灵活而明确的调用签名；对应用开发者来说只是偶

尔用到，但读别人的代码时几乎天天遇到。

·底层实现：Python 在调用时构造一个"参数分配器" (argument clinic)，按顺序执行解包、按位置分配、按名字匹配、收集剩余、填默认值等步骤（详见 18.9 节的七步算法），全部完成后才开始执行函数体。

·实际问题：print、sorted、dict、str.format 等内置函数全部依赖这些模式；你写的类、装饰器、回调函数也会用到。

·初学者误区：把这些模式当成必须掌握的日常语法。其实默认的位置匹配就够用了；特殊模式是"工具箱"，不是"日常工具"。

18.7 参数匹配概览 (Argument Matching Overview)

英文原文

Before we get into syntax details, it's important to stress that these special modes are optional and deal only with matching objects to names; the underlying passing mechanism after the matching takes place is still assignment. In fact, some of these tools are intended more for people writing libraries than for application developers. That said, you may stumble across these modes even if you don't code them yourself, so the following summarizes all the options: Positionals: matched from left to right. The normal case, which w

e've mostly been using so far, is to match argument values (passed in a call) to argument names (listed in a function definition) by position, from left to right. Keywords: matched by argument name. Alternatively, callers can explicitly specify which argument in the function is to receive a value, by giving the definition's name for the argument with `name=value` syntax. Defaults: specify values for optional arguments that aren't passed. Functions themselves can specify default values for arguments to receive if the call passes too few values, again using the `name=value` syntax. Starred collectors: collect arbitrarily many positional or keyword arguments. Function definitions can use special arguments preceded with one or two characters to collect an arbitrary number of arguments after other matching. This feature is sometimes referred to as `varargs`, after a variable-length argument list tool in the C language; in Python, the arguments are collected in a normal object. Starred unpackers: pass arbitrarily many positional or keyword arguments. Function calls can also use the one or two syntax to unpack argument collections into separate arguments. This is the inverse of `a` in a function definition—in the definition it means collect arbitrarily many arguments, while in the call it means unpack arbitrarily many arguments, and pass them individually as discrete values. Keyword-only arguments: arguments that must be passed by name. Function definitions can also use `a` in their header

rs to specify arguments that must be passed by name as keyword arguments, not position. This is often used for configuration options that augment primary arguments. Positional-only arguments: arguments that must be passed by position. As of Python 3.8, function definitions may additionally use a / in their header's arguments list to specify that all arguments preceding it must be passed by position, not keyword-argument name.

中文翻译

在进入语法细节之前，必须强调：这些特殊模式是可选的，它们只处理“把对象匹配到名字”这件事；匹配完成之后的底层传递机制仍然是赋值。事实上，其中一些工具更多是为写库的人准备的，而不是为应用开发者。话虽如此，即使你自己不写这些模式，也可能会撞见它们，所以下面总结了全部选项：位置参数（Positionals）：从左到右匹配。常规情况，也是我们到目前为止用得最多的——把实参值（调用中传入）按位置、从左到右匹配到形参名（函数定义中列出）。关键字参数（Keywords）：按参数名匹配。或者，调用方可以用 `name=value` 语法给出定义中的参数名，显式指定函数中哪个参数接收这个值。默认值（Defaults）：为未传入的可选参数指定值。函数本身可以指定默认值：如果调用传入的值太少，这些参数就接收默认值，同样使用 `name=value` 语法。星号收集器（Starred collectors）：收集任意数量的位置或关键字参数。函数定义可以用一个或两个前缀的特殊参数，在完成其他匹配后收集任意数量的参数。这个特性有时被称为 `varargs`（可变参数），源自 C 语

言中一种变长参数列表工具；在 Python 中，参数被收集进一个普通对象。星号解包器（Starred unpackers）：传入任意数量的位置或关键字参数。函数调用也可以用一星或两星语法，把参数集合解包成单独的参数。这与函数定义中的互为逆运算——在定义中它表示"收集任意多参数"，而在调用中它表示"解包任意多参数，把它们作为离散值逐一传入"。仅关键字参数（Keyword-only arguments）：必须按名字传递的参数。函数定义还可以在函数头中使用一个，指定某些参数必须以关键字参数的形式按名字传递，而不能按位置传递。这通常用于为主参数锦上添花的配置选项（configuration options）。仅位置参数（Positional-only arguments）：必须按位置传递的参数。从 Python 3.8 开始，函数定义还可以在参数列表头中使用一个 /，指定它前面的所有参数必须以位置传递，而不能用关键字参数名传递。

深度理解

- 核心概念：六种匹配模式全家福——位置、关键字、默认值、星号收集（定义侧）、星号解包（调用侧）、仅关键字、仅位置。本章余下部分逐一展开。
- 术语对照：varargs 来自 C 语言的 printf 风格变长参数；Python 的 args/kwargs 是它的"收集"版本，但收集结果不是 C 的裸指针栈，而是普通元组/字典对象——可以索引、可以遍历、可以传给别的函数。
- 为什么这样设计：把"灵活性"留给签名（signature），把"确定性"留给实现。库作者通过签名告诉调用方"你怎么调"，内部则用统一的赋值机制处理。
- 设计思想：/ 是 Python 3.8（PEP 570）加入的，（仅关

键字) 是 Python 3.0 (PEP 3102) 加入的; 用于调用侧解包在 Python 3.5 (PEP 448) 扩展到多个。这章描述的其实是 30 多年语法演进的"汇总快照"。

18.8 参数匹配语法 (Argument Matching Syntax)

英文原文

As a reference and preview, Table 18-1 summarizes the syntax that invokes the argument-matching modes. Table 18-1. Function argument-matching forms

Syntax	Location	Interpretation
func(value)	Caller	Normal argument: matched by position
func(name=value)	Caller	Keyword argument: matched by name
func(iterable)	Caller	Pass all objects in iterable as individual positional argument
func(dict)	Caller	Pass all key/value pairs in dict as individual keyword arguments
def func(name)	Function	Normal argument: matches any passed value by position or
def func(name=value)	Function	Default argument value, if not passed in the call
def func(name)	Function	Matches and collects remaining positional arguments in a tuple

<code>def func(name)</code>	Function	Matches and collects remaining keyword arguments in a dictionary
<code>def func(name, name)</code>	Function	Arguments that must be passed by keyword only in calls
<code>def func(, name)</code>	Function	Arguments that must be passed by keyword only in calls
<code>def func(name, /)</code>	Function	Arguments that must be passed by position only in calls

The argument-matching modes listed in this table break down between function calls and definitions as follows: In a function call (the first four rows of the table): Simple values are matched to definition arguments by position, but using the `name=value` form tells Python to match arguments by name instead; these are called keyword arguments. Any number of iterable or dict forms can be used in a call, allowing us to bundle many positional or keyword objects in iterables and mappings, respectively, and unpack them as separate, individual arguments when they are passed to the function. In a function definition (the rest of the table): A simple name is matched by position or name depending on how the caller passes it, but the `name=value` form specifies a default value. The `name` form collects any extra unmatched positional arguments in a tuple, and `name` collects unmatched keyword arguments in a dictionary. In addition, any normal or defaulted argument names following a `name` or a bare `ar`

e keyword-only arguments and must be passed by keyword in calls, and arguments preceding a / are positional-only arguments that must not be passed by keyword name. Of these, keyword arguments and defaults are probably the most commonly used in Python code. We've informally used both of these earlier in this book: We've already used keywords to specify options to the print function, but they are more general—keywords allow us to label any argument with its name, to make calls more explicit and informational. We met defaults earlier, too, as a way to pass in values from the enclosing function's scope, but they are also more general—they allow us to make any argument optional, providing its default value in a function definition. As you'll see ahead, the combination of defaults in a function definition and keywords in a call further allows us to pick and choose which defaults to override per call. In short, special argument-matching modes let you be fairly liberal about how many arguments must be passed to a function. If a function specifies defaults, they are used if you pass too few arguments. If a function uses the argument-collector forms, you can seemingly pass too many arguments; the names collect the extra arguments in data structures for processing within the function.

中文翻译

作为参考和预览，表 18-1 汇总了调用参数匹配模式的语法

。

表 18-1. 函数参数匹配形式

语法	位置	含义
<code>func(value)</code>	调用方	普通参数：按位置匹配
<code>func(name=value)</code>	调用方	关键字参数：按名字匹配
<code>func(iterable)</code>	调用方	把 <code>iterable</code> 中的所有对象作为独立的位置参数传入
<code>func(dict)</code>	调用方	把 <code>dict</code> 中的所有键/值对作为独立的关键字参数传入
<code>def func(name)</code>	函数	普通参数：匹配任何传入的值，按位置或名字
<code>def func(name=value)</code>	函数	默认参数值：调用未传入时使用
<code>def func(name)</code>	函数	匹配并收集剩余的位置参数到一个元组
<code>def func(name)</code>	函数	匹配并收集剩余的关键字参数到一个字典
<code>def func(name, name)</code>	函数	调用中必须仅以关键字方式传递的参数
<code>def func(, name)</code>	函数	调用中必须仅以关键字方式传递的参数
<code>def func(name, /)</code>	函数	调用中必须仅以位置方式传递的参数

表中列出的参数匹配模式，在函数调用与定义之间这样划分：在函数调用中（表的前四行）：简单值按位置匹配到定义中的参数，但使用 `name=value` 形式则是告诉 Python 改按名字匹配——这些被称为关键字参数。调用中可以使用任意数量的 `iterable` 或 `dict` 形式，分别让我们把许多位置或关键字对象打包进可迭代对象和映射，并在传给函数时把它们解包成独立、单独的实参。在函数定义中（表的其余行

)：简单的 name 根据调用方传入方式按位置或名字匹配，但 name=value 形式指定的是默认值。name 形式把任何额外未匹配的位置参数收集进一个元组，name 把未匹配的关键字参数收集进一个字典。此外，跟在 name 或裸之后的任何普通或带默认值的参数名都是仅关键字 (keyword-only) 参数，调用中必须以关键字传入；而位于 / 之前的参数是仅位置 (positional-only) 参数，绝不能以关键字名字传入。其中，关键字参数和默认值大概是 Python 代码中最常用的。本书前面我们已经非正式地用过这两者：我们已经用关键字给 print 函数指定过选项，但它们更通用——关键字允许我们给任何参数贴上名字标签，使调用更显式、信息量更大。我们之前也遇到过默认值，用它从外层函数作用域传入值；但它们也更通用——它们让我们可以在函数定义中提供默认值，从而让任何参数变成可选的。你将在后面看到，函数定义中的默认值与调用中的关键字相结合，还能让我们在每次调用中挑选要覆盖哪些默认值。简而言之，特殊参数匹配模式让你在“函数必须传入多少个参数”上相当宽松。如果函数指定了默认值，你传少了就用默认值；如果函数使用了参数收集形式，你甚至看起来可以传“太多”参数——名字会把多余的参数收集进数据结构，供函数内部处理。

深度理解

·核心概念：表 18-1 是本章的“总纲”，11 种语法形式分布在“调用侧”和“定义侧”两个战场。务必记住：同名符号在不同位置含义相反——在定义里是“收集”，在调用里是“解包”；name=value 在定义里是“默认值”，在调用里是“关键字参数”。

·底层实现：定义侧语法在编译期被解析进函数对象的 `code.covarnames` 与参数标志位（`COVARARGS/COVARKEYWORDS`）；调用侧语法在字节码层面生成 `LOAD` 后直接压栈（简单值）、`BUILDTUPLEUNPACK/BUILDMAPUNPACK`（星号解包）。

·为什么这样设计：位置匹配是默认零成本方案；关键字提升可读性；/ 解决"参数个数未知"；/ 与解决"传递模式必须受限"——每件工具解决一类实际问题。

·生活类比：点餐单——位置参数就是"按顺序上菜"（第一个菜给第一个格子），关键字就是"写明'可乐加冰'"（点名要哪个选项），`dict` 就是"整张预填好的表格直接塞进去"。

·初学者误区：把 `def f(name=value)` 的默认值语法和 `f(name=value)` 的关键字语法搞混——长得一样，含义完全相反；以及以为 `args` 必须在所有函数里出现——它只是可选工具。

18.9 参数传递细节 (Argument Passing Details)

英文原文

If you choose to use and combine the special argument-matching modes, Python will ask you to follow some ordering rules among the modes' optional components. We're going to defer full and formal coverage

e of these until we've had a chance to observe these modes in action, but as some initial tips: In a function call, any number of positionals, keywords, and starred unpackings can be used, but positional arguments must precede keyword arguments and dict unpackings, and iterable unpackings must precede dict unpackings. In a function definition, arguments must appear in this order: any positional-only arguments; followed by any positional-or-keyword arguments; followed by the optional name positional collector or and any keyword-only arguments; followed by the optional name keyword collector. Arguments can have optional defaults (name=value), but once a default is used, all arguments must use defaults up to a , after which keyword-only arguments allow defaults and nondefaults to be freely mixed. If you mix arguments in any other order, you will get a syntax error because the combinations can be ambiguous. The steps that Python internally carries out to match arguments before assignment can roughly be described as follows: 1. Unpack all args at the call into nonkeyword arguments. 2. Unpack all args at the call into keyword arguments. 3. Assign nonkeyword arguments by position. 4. Assign keyword arguments by matching names. 5. Collect extra nonkeyword arguments in the name tuple. 6. Collect extra keyword arguments in the name dictionary. 7. Assign default values to unassigned arguments. After arguments in call and definition are matched, P

python checks to make sure each argument is passed just one value (or else an error is raised) and then assigns argument names to the objects passed to them and runs the function body. The actual matching algorithm Python uses is a bit more complex, so we'll defer to Python's standard language manual for a more exact description. It's not required reading, but tracing Python's matching algorithm may help you to understand some convoluted cases, especially when modes are mixed. We'll return to the ordering rules in function calls in definitions with higher fidelity after we've had a chance to meet all the players. Let's get started in the next section with the most common of the bunch.

NOTE But annotations are moot: Argument names in a `def` statement (only) can also have annotation values, specified as `name: annot`, or `name: annot = default` when defaults are present. This is simply additional syntax for arguments and does not augment or change argument-ordering rules. The function itself can also have an annotation value, given as `def f(...)->annot`, and Python attaches all annotation values to the function object. See the discussion of function annotation in Chapter 19 for more details, and the section on their role in unused type hinting in Chapter 6.

中文翻译

如果你选择使用并组合这些特殊参数匹配模式，Python 会要求你遵守各模式可选组件之间的排序规则（ordering rules）。我们先把这些规则的完整正式版推迟到后面、等我们有机会亲眼看到这些模式发挥作用之后再说，但先给一些初步提示：在函数调用中，位置参数、关键字参数和星号解包可以用任意数量，但位置参数必须位于关键字参数和 dict 解包之前，而且 iterable 解包必须位于 dict 解包之前。在函数定义中，参数必须按这个顺序出现：先任何仅位置参数；接着任何"位置或关键字"参数；接着可选的 name 位置收集器或裸，以及任何仅关键字参数；最后是可选的 name 关键字收集器。参数可以有可选默认值（name=value），但一旦使用了默认值，在之前所有参数都必须使用默认值；之后，仅关键字参数允许默认值和非默认值自由混排。如果以任何其他顺序混排参数，你会得到语法错误（syntax error），因为组合可能产生歧义。Python 在赋值之前内部执行的参数匹配步骤，大致可以描述如下：1. 解包调用中所有的 args，变成非关键字参数。2. 解包调用中所有的 args，变成关键字参数。3. 按位置分配非关键字参数。4. 按名字匹配分配关键字参数。5. 把多余的非关键字参数收集进 name 元组。6. 把多余的关键字参数收集进 name 字典。7. 给未赋值的参数分配默认值。调用与定义中的参数匹配完成后，Python 会检查每个参数是否恰好被传入一个值（否则抛出错误），然后把参数名赋值为传给它们的对象，并运行函数体。Python 实际使用的匹配算法比这稍微复杂一些，更精确的描述请参阅 Python 标准语言手册。这不是必读内容，但追踪 Python 的匹配算法可能有助于你理解一些错综

复杂的情况，尤其是模式混合使用的时候。我们将在见过所有"角色"之后，以更高保真度回到调用与定义中的排序规则。让我们从下一节开始，认识其中最常用的一批。

注意 (NOTE) 但注解是无关紧要的 (annotations are moot) : def 语句 (仅限它) 中的参数名还可以带有注解 (annotation) 值，写法是 name: annot，有默认值时是 name: annot = default。这只是参数的附加语法，不会增加或改变参数排序规则。函数本身也可以有注解值，写法是 def f(...)->annot，Python 会把所有注解值附加到函数对象上。更多细节见第 19 章函数注解的讨论，以及第 6 章中关于它们在"未使用的类型提示"中角色的那一节。

深度理解

·核心概念：七步匹配算法是 Python 参数处理的"心电图"——解包 → 按位置分配 → 按名字分配 → 收集 → 填默认值。所有复杂案例都可以套用这个流程推导结果。

·底层实现：这七步在 CPython 的 CALLFUNCTIONEX/PyEvalEvalFrameEx 与 PyArgParser 机制中实现；函数对象在编译期就解析好参数种类 (coflags 中的 COVARARGS/COVARKEYWORDS)，运行期只做"填槽"。

·为什么这样设计：排序规则的存在是因为语法必须无歧义——一旦允许"位置参数跟在关键字参数后面"，就无法确定某个值该给谁。规则把歧义扼杀在编译期 (SyntaxError)。

·实际问题：调用 dict(mapping)、设计 init 签名、写装饰器转发参数——理解七步算法后，这些都不再需要死记

硬背。

·初学者误区：不区分"定义侧"与"调用侧"的规则——两侧语法相似但规则完全不同；另外要注意"每个参数只能有一个值"的检查发生在赋值之前，重复赋值会直接报 `TypeError`。

·生活类比：七步算法像流水线分拣：先把集装箱（args）拆开、把标注好的盒子（kwargs）拆开，然后按地址投递（位置、名字），剩不下就暂存（收集），地址不全就用备用方案（默认值）。

18.10 关键字与默认值示例 (Keyword and Default Examples)

英文原文

Argument passing is simpler in code than the preceding descriptions may imply. First off, if you don't use any special matching syntax, Python matches names by position from left to right. For instance, if you define a function that requires three arguments, you must call it with three arguments: `python >>> def f(a, b, c): print(a, b, c) >>> f(1, 2, 3)` 1 2 3 Here, we pass by position—a is matched to 1, b is matched to 2, and so on. This works like it does in most other programming languages.

中文翻译

参数传递在代码里比前面这些描述看起来要简单得多。首先，如果你不使用任何特殊匹配语法，Python 按位置从左到右匹配名字。例如，如果你定义一个需要三个参数的函数，就必须用三个参数调用它：`python >>> def f(a, b, c): print(a, b, c) >>> f(1, 2, 3)` 1 2 3 这里我们按位置传递——a 匹配 1，b 匹配 2，依此类推。这与大多数其他编程语言的工作方式一样。

深度理解

·核心概念：位置匹配是默认模式，也是零配置模式——函数的第 N 个形参接收调用的第 N 个实参。

·底层实现：`f(1, 2, 3)` 编译为三条 `LOADCONST`（依次压入 1、2、3）加一条 `CALL`。新帧的局部槽 a、b、c 依次填入这三个对象——纯粹按序操作，不做任何名字查找。

·为什么这样设计：最直观、最快，符合几乎所有语言的直觉，是“零语法”的默认行为。

·初学者误区：位置参数个数必须精确匹配——`f(1, 2)` 或 `f(1, 2, 3, 4)` 都会抛 `TypeError`，Python 不做隐式丢弃或填充。

18.10.1 关键字 (Keywords)

英文原文

In Python, though, you can be more specific about what goes where when you call a function. Keyword ar

guments allow us to match by name, instead of by position. Using the same function definition but a different call:

```
python >>> f(c=3, b=2, a=1) 1 2 3
```

The `c=3` in this call, for example, means send 3 to the argument named `c`. More formally, Python matches the name `c` in the call to the argument named `c` in the function definition, and then assigns the value 3 to that argument. The net effect of this call is the same as that of the prior call, but notice that the left-to-right order of the arguments no longer matters when keywords are used because arguments are matched by name, not by position. It's even possible to combine positional and keyword arguments in a single call. In this case, all positionals are matched first from left to right in the definition, before keywords are matched by name:

```
python >>> f(1, c=3, b=2) # a gets 1 by position, b and c passed by name 1 2 3
```

When most people see this the first time, they wonder why one would use such a tool. Keywords typically have two roles in Python. First, they make your calls a bit more self-documenting (assuming that you use better argument names than `a`, `b`, and `c`!). For example, a call of this form:

```
python func(title='Learning Python', edition=6, year=2024, python=3.12)
```


is much more meaningful than a call with three naked values separated by commas, especially in larger programs—the keywords serve as labels for the data in the call:

```
python func(title='Learning Python', 6, 2024, 3.12)
```

The second major use of keywords occurs in conjunction with defaults, which we turn to next.

中文翻译

不过，在 Python 中，调用函数时你可以更精确地指定“什么值去哪”。关键字参数允许我们按名字匹配，而不是按位置。用同一个函数定义但换一种调用方式：`python >>> f(c=3, b=2, a=1)` 1 2 3 例如，这个调用中的 `c=3` 意思是“把 3 送给名为 `c` 的参数”。更正式地说，Python 把调用中的名字 `c` 与函数定义中名为 `c` 的参数匹配，然后把值 3 赋给那个参数。这个调用的净效果与上一个调用相同，但注意：使用关键字时，参数的从左到右顺序不再重要，因为参数是按名字、而不是按位置匹配的。甚至可以在一次调用中组合位置参数和关键字参数。这种情况下，所有位置参数先在定义中从左到右匹配完毕，然后关键字参数才按名字匹配：`python >>> f(1, c=3, b=2)` # `a` 按位置得到 1，`b` 和 `c` 按名字传入 1 2 3 大多数人第一次看到这个时都会好奇：为什么要用这种工具？关键字在 Python 中通常有两个角色。第一，它们让你的调用更“自文档化”（前提是你使用比 `a`、`b`、`c` 更好的参数名！）。例如，这样形式的调用：`python func(title='Learning Python', edition=6, year=2024, python=3.12)` 比三个用逗号分隔的裸值要有意义得多，尤其是在较大的程序中——关键字充当了调用中数据的标签：`p`

python func(title='Learning Python', 6, 2024, 3.12) 关键字的第二个主要用途是与默认值配合使用，我们马上就会讲到。

深度理解

- 核心概念：关键字参数 = 按名匹配；与位置参数混用时，位置参数先行，关键字随后按名匹配。
- 底层实现：带关键字参数的调用，在字节码层面把关键字名存为 LOADCONST 的字符串，CALLFUNCTIONEX 或 CALL 指令把"名字→值"配对交给函数对象；运行时匹配器（PyArgParser）按名字到形参表中查找对应槽位。
- 为什么这样设计：自文档化（self-documenting）——`f`unc(title='...', edition=6, ...) 读起来像一份清单；并且它把"顺序耦合"解除，调用方不必记忆参数顺序。
- 实际问题：GUI 框架（如 tkinter 的 `Button(text=..., command=...)`）、`sorted(iterable, key=..., reverse=...)`、`print(..., sep=..., end=...)`——配置项多时，关键字是唯一不靠顺序的解法。
- 初学者误区：把关键字参数误认为"可以随便放在任何位置"——不，位置参数必须全部在关键字参数之前（见 18.9 节规则）；`f(1, c=3, b=2)` 合法，`f(c=3, 1, b=2)` 是 `SyntaxError`。

18.10.2 默认值 (Defaults)

英文原文

We talked about defaults in brief earlier, when discussing nested function scopes. In short, defaults allow us to make selected function arguments optional; if not passed a value, the argument is assigned its default before the function runs. For example, here is a function that requires one argument and defaults two others:

```
python >>> def f(a, b=2, c=3): print(a, b, c) # a required, b and c optional
```

As noted earlier, defaults must appear after nondefaults at this point in a header (they can be mixed after `a` as you'll see ahead). When we call this function, we must provide a value for `a`, either by position or by keyword; however, providing values for `b` and `c` is optional. If we don't pass values to `b` and `c`, they default to 2 and 3, respectively:

```
python >>> f(1) # Use defaults 1 2 3
>>> f(a=1) 1 2 3
```

If we pass two values, only `c` gets its default, and when passing three values, no defaults are used:

```
python >>> f(1, 4) # Override defaults 1 4 3
>>> f(1, 4, 5) 1 4 5
```

Finally, here is how the keyword and default features interact. Because they subvert the normal left-to-right positional mapping, keywords allow us to essentially

y skip over arguments with defaults:

```
python >>> f(1, c=6) # Choose defaults: b in the middle 1 2 6
```

Here, a gets 1 by position, c gets 6 by keyword, and b, in between, defaults to 2. Be careful not to confuse the special name=value syntax in a function definition and a function call; in the call it means a match-by-name keyword argument, while in the definition it specifies a default for an optional argument. In both cases, this is not an assignment statement (despite its appearance); it is special syntax for these two contexts, which modifies the default argument-matching mechanics.

中文翻译

我们在前面讨论嵌套函数作用域时简要提到过默认值。简言之，默认值让我们可以把选定的函数参数变成可选的（optional）；如果没有传值，该参数会在函数运行前被赋予其默认值。例如，下面这个函数需要一个参数，并为另外两个参数提供默认值：python >>> def f(a, b=2, c=3): print(a, b, c) # a 必填，b 和 c 可选如前所述，在这个位置（之前），默认值必须出现在非默认值之后（稍后你会看到，之后可以混排）。调用这个函数时，我们必须为 a 提供值，无论按位置还是按关键字；而为 b 和 c 提供值则是可选的。如果我们不向 b 和 c 传值，它们分别默认为 2 和 3：python >>> f(1) # 使用默认值 1 2 3 >>> f(a=1) 1 2 3 如果我们传两个值，只有 c 使用默认值；传三个值时，不使用任何默认值：python >>> f(1, 4) # 覆盖默认值 1 4 3 >>> f

(1, 4, 5) 1 4 5 最后，看看关键字与默认值特性如何互动。由于关键字绕过了正常的从左到右位置映射，它们让我们可以"跳过"带默认值的参数：`python >>> f(1, c=6)` # 挑着用默认值：b 在中间 1 2 6 这里，a 按位置得到 1，c 按关键字得到 6，而夹在中间的 b 默认为 2。注意不要把函数定义与函数调用中的特殊 `name=value` 语法搞混；在调用中它表示按名匹配的关键字参数，而在定义中它为一个可选参数指定默认值。在两种情况下，它都不是赋值语句（尽管看起来像）；它是这两个上下文中各自的特殊语法，用于修改默认的参数匹配机制。

深度理解

·核心概念：默认值是"定义侧的可选开关"——`def f(a, b=2, c=3)` 里 b、c 变可选；关键字是"调用侧的精确指定"——两者组合后可以跳过中间的默认参数。

·底层实现：`def f(a, b=2, c=3)` 编译时，默认值表达式在定义时刻被求值一次，结果对象存放在函数对象的 `defaults` 元组里；每次调用缺参数时，匹配器直接取 `defaults` 里的对象填入槽位。注意：默认值是"定义时求值一次、调用时复用"——这是后面可变默认值陷阱的根源。

·为什么这样设计：`f(1, c=6)` 允许"跳过 b"是因为关键字按名匹配，与位置无关；这让 API 可以保留"中间参数也有默认值"的宽松度。

·生活类比：食堂套餐——a 是必选主食，b 是默认小菜、c 是默认饮料；你说"一份主食，饮料要可乐" (`f(1, c=6)`)，小菜就上默认款。

·初学者误区：① 把 `def f(a, b=2, c=3)` 里 `b=2` 理解成"每次调用都重新赋值为 2"——错，默认值对象只创建一次（见下节 NOTE）；② 默认值必须跟在非默认值后面——`def f(a=1, b)` 是 `SyntaxError`。

18.10.3 组合关键字与默认值 (Combining keywords and defaults)

英文原文

Here is a slightly larger example that demonstrates keywords and defaults in action. In the following, the caller must always pass at least two arguments (to match code and hack), but the other two are optional. If they are omitted, Python assigns script and app to the defaults specified in the definition:

```
python def func(code, hack, script=0, app=0): # First
2 required print((code, hack, script, app)) func(1, 2) #
Output: (1, 2, 0, 0) func(1, app=1, hack=0) # Output: (
1, 0, 0, 1) func(code=1, hack=0) # Output: (1, 0, 0, 0) f
unc(script=1, hack=2, code=3) # Output: (3, 2, 1, 0) f
unc(1, 2, 3, 4) # Output: (1, 2, 3, 4)
```

Notice again that when keyword arguments are used in the call, the order in which the arguments are listed doesn't matter; Python matches by name, not by position. The caller must supply values for code and hack, but they can be given by position or by name.

Also keep in mind that all the special definition-side argument-matching syntax we're exploring in this chapter works the same in `def` and `lambda`, though most examples use the former, and the latter returns results implicitly:

```
python >> func = lambda code, hack, script=0, app=0: (code, hack, script, app) >> func(1, 2) (1, 2, 0, 0) > > func(script=1, hack=2, code=3) (3, 2, 1, 0)
```

NOTE Beware mutable defaults: As noted in the prior chapter, if you code a default to be a mutable object (e.g., `def f(a=[])`), the same, single mutable object is reused every time the function is later called—even if it is changed in place within the function. The net effect is that the argument's default retains its value from the prior call and is not reset to its original value coded in the function header. To reset anew on each call, move the assignment into the function body instead. Mutable defaults allow state retention, but this is often an unpleasant surprise. Since this is such a common trap, we'll postpone further exploration until this part's "gotchas" list at the end of Chapter 21.

中文翻译

下面是一个稍大的例子，演示关键字与默认值的实战配合。在下面的代码中，调用方必须至少传入两个参数（匹配 `code` 和 `hack`），另外两个是可选的。如果省略，Python 会把 `script` 和 `app` 赋为定义中指定的默认值：

```
python def func(code, hack, script=0, app=0): # 前 2 个必填 print((
```

code, hack, script, app)) func(1, 2) # 输出: (1, 2, 0, 0) func(1, app=1, hack=0) # 输出: (1, 0, 0, 1) func(code=1, hack=0) # 输出: (1, 0, 0, 0) func(script=1, hack=2, code=3) # 输出: (3, 2, 1, 0) func(1, 2, 3, 4) # 输出: (1, 2, 3, 4) 再次注意: 当调用中使用关键字参数时, 参数的列出顺序无关紧要; Python 按名字、而不是按位置匹配。调用方必须为 code 和 hack 提供值, 但它们可以按位置或按名字给出。还要记住: 本章探索的所有定义侧的特殊参数匹配语法, 在 def 和 lambda 中工作方式完全相同, 只是大多数示例使用前者, 而后者是隐式返回结果: python >>> func = lambda code, hack, script=0, app=0: (code, hack, script, app) >>> func(1, 2) (1, 2, 0, 0) >>> func(script=1, hack=2, code=3) (3, 2, 1, 0)

注意 (NOTE) 当心可变默认值 (mutable defaults): 如上一章所述, 如果你把默认值写成一个可变对象 (如 def f(a=[])), 那么每次调用这个函数时, 复用的是同一个可变对象——即使它在函数内被原地修改过。净效果是: 该参数的默认值会保留上一次调用时的状态, 而不会重置为函数头中写下的原始值。要想每次调用都重新初始化, 就把赋值挪进函数体。可变默认值允许状态留存, 但这常常是一个令人不快的意外。由于这是如此常见的陷阱, 我们把它推迟到本部分末尾第 21 章的"陷阱" (gotchas) 清单中再深入探讨。

深度理解

·核心概念: 关键字 + 默认值 = "任意挑着覆盖"——调用方只指定关心的参数, 其余交给默认值。

·底层实现：默认值对象在 `def` 语句执行时求值并存入函数对象的 `defaults` 元组。`def f(a=[])` 中的 `[]` 在定义那一刻创建一次——不是每次调用创建。这就是陷阱的根源：所有调用共享同一个列表对象。

·陷阱演示：`def add(x, lst=[]): lst.append(x); return lst`——连续调用会不断累积：`add(1) → [1]`，`add(2) → [1, 2]`。正确的写法是 `def add(x, lst=None): if lst is None: lst = []`。

·为什么这样设计：这是 CPython 的实现方式（默认值求值一次）的自然结果，官方称其为“设计缺陷但被刻意保留”（Python FAQ 明确说过不会修）；`None` 哨兵法是最佳实践。

·实际问题：库作者经常用 `None` 哨兵 + 函数体内建新对象；而“每次调用保留状态”有时被刻意利用（如记忆化缓存、计数器），但绝大多数情况下是 bug。

·生活类比：默认值对象像“公用的公共碗筷”——每次来客人（调用）都用同一套，上一桌用过的油渍（上次修改）还留在上面。想每次干净，就每次自带一次性碗筷（`None` 哨兵 + 函数内新建）。

代码分析

```
def func(code, hack, script=0, app=0):    # 2
    print((code, hack, script, app))
func(1, app=1, hack=0)                   # (1, 0, 0, 1)
```

解释器如何处理 `func(1, app=1, hack=0)`：

1. 匹配开始：位置参数 1 先按位置进入 `code`（第 3 步七

步算法)。

2. 关键字 `hack=0` 按名字匹配 `hack`, `app=1` 按名字匹配 `app`。

3. `script` 未被赋值, 步骤 7 从 `defaults` 取 0 填入。

4. 全部参数各就各位后, 解释器才执行函数体 `print(...)`, 输出 `(1, 0, 0, 1)`。

设计思想: `code`、`hack` 是"主参数" (必须给), `script`、`app` 是"配置参数" (可选) ——这种"主参数 + 可选配置"的签名模式在 Python 标准库中极其常见, 也是 18.13 节仅关键字参数诞生的动机。

18.11 任意参数示例 (Arbitrary Arguments Examples)

英文原文

The last two argument-matching extensions, `and` and `*`, are designed to support any number of arguments. Both can appear in either the function definition or a function call, and they have related purposes in the two locations.

中文翻译

最后两个参数匹配扩展——`and`和`*`——被设计用来支持任意数量的参数。两者都可以出现在函数定义或函数调用中, 而且在这两个位置上它们的目的相互关联。

深度理解

- 核心概念：和是"数量未知"问题的答案。定义侧负责"收" (collect)，调用侧负责"撒" (unpack) ——一个功能，两种用法，符号相同但方向相反。
- 底层实现：函数对象通过 `coflags` 中的 `COVARARGS` (`args`) 与 `COVARKEYWORDS` (`kwargs`) 标记记录自己是否带收集器；调用时多出的参数要么 `BUILDTUPLE` 成元组，要么 `BUILDMAP` 成字典。
- 为什么这样设计：C 语言的 `varargs` (变长参数) 先例 + Python"一切皆对象"哲学的结合：收集结果不是裸指针，而是普通元组/字典，可以直接用 Python 工具处理。
- 初学者误区：以为 `args` 是"数组"——它是元组；以为 `kwargs` 里的键会丢失顺序——Python 3.6+ 字典保持插入序，收集器的键序与调用时关键字顺序一致。

18.11.1 定义中：收集参数 (Definitions: Collecting arguments)

英文原文

The first use, in the function definition, collects unmatched positional arguments into a tuple:

```
python >> def f(args): print(args)
```

When this function is called, Python collects all the positional arguments into a new tuple and assigns the variable `args` to that tuple. Because it is a normal tuple

e object, it can be indexed, stepped through with a for loop, and so on:

```
python >> f() () >> f(1) (1,) >> f(1, 2, 3, 4) (1, 2, 3, 4)
```

The feature is similar, but it only works for keyword arguments—it collects them into a new dictionary, which can then be processed with normal dictionary tools. In a sense, the form allows you to convert from keywords to dictionaries, which you can then step through with keys calls, dictionary iterators, and the like (this is roughly what the dict call does when passed keywords, but it returns the new dictionary):

```
python >> def f(args): print(args) >> f() {} >> f(a=1, b=2) {'a': 1, 'b': 2}
```

Finally, function definitions can combine normal arguments, the `*`, and the `**` to implement wildly flexible call signatures. For instance, in the following, `1` is passed to `a` by position, `2` and `3` are collected into the `pargs` positional tuple, and `x` and `y` wind up in the `kargs` keyword dictionary:

```
python >> def f(a, pargs, kargs): print(a, pargs, kargs) >> f(1, 2, 3, x=1, y=2) 1 (2, 3) {'x': 1, 'y': 2}
```

Notice that the dictionary's keys are ordered here: as of Python 3.6, the collector's keys preserve the order in which keyword arguments were passed to the function. This relies on the insertion order of keys in dictionaries at large (see Chapter 8 for a refresher if you've

e forgotten what that means).

Functions with both `and` may be rare, but they show up in functions that need to support multiple call patterns (for backward compatibility, for instance). In fact, these features can be combined in even more complex ways that may seem ambiguous at first glance—an idea we will revisit later in this chapter when we lock down ordering rules. First, though, let's see what happens when `and` are coded in function calls instead of definitions.

中文翻译

第一种用途，在函数定义中，把未匹配的位置参数收集进一个元组：python >>> def f(args): print(args) 当这个函数被调用时，Python 把所有位置参数收集进一个新元组，并把变量 `args` 赋值为该元组。因为它是一个普通元组对象，所以可以被索引、可以用 `for` 循环遍历，等等：python >>> f() () >>> f(1) (1,) >>> f(1, 2, 3, 4) (1, 2, 3, 4) 特性类似，但它只对关键字参数有效——它把这些关键字收集进一个新字典，之后可以用普通字典工具处理。在某种意义上，形式让你把关键字转换成字典，然后你可以用 `keys` 调用、字典迭代器等遍历它（这大致就是 `dict` 调用在传入关键字时做的事，只不过它返回新的字典）：python >>> def f(args): print(args) >>> f() {} >>> f(a=1, b=2) {'a': 1, 'b': 2} 最后，函数定义可以组合普通参数、`and`，实现极其灵活的调用签名。例如，在下面的代码中，1 按位置传给 `a`，2 和 3 被收集进 `pargs` 位置元组，`x` 和 `y` 则进了 `kargs` 关键字字典：python >>> def f(a, pargs, kargs): print(a

, pargs, kargs) >>> f(1, 2, 3, x=1, y=2) 1 (2, 3) {'x': 1, 'y': 2} 注意这里的字典键是有序的：从 Python 3.6 起，收集器的键保留关键字参数传给函数的顺序。这依赖于字典整体上按键的插入序（如果你忘了这意味着什么，翻回第 8 章复习一下）。同时带和的函数可能少见，但它们出现在需要支持多种调用模式的函数中（比如为了向后兼容）。事实上，这些特性还能以更复杂的方式组合，乍看之下似乎有歧义——我们会在本章后面锁定排序规则时再回到这个想法。不过首先，让我们看看当和被写在函数调用中而不是定义中时，会发生什么。

深度理解

- 核心概念：args 收集"多余位置参数"成元组；kwargs 收集"多余关键字参数"成字典；两者可与普通参数共存，形成 f(a, pargs, kargs) 的"全能签名"。
- 底层实现：f(1, 2, 3, x=1, y=2) 的调用指令会把位置与关键字分开打包；进入函数帧时，匹配器按七步算法分配后，把剩余位置参数 (2, 3) 用 BUILD_TUPLE 建元组给 pargs，剩余关键字用字典构造给 kargs。键序保留来自 Python 3.6 起的字典插入序实现（PEP 468 保证 kwargs 有序）。
- 为什么这样设计：向后兼容包装器（wrapper）、装饰器、日志/追踪器、以及"把参数原样转发给另一个函数"的需求——没有 args, kwargs，这些都需要手写所有组合。
- 实际问题：functools.wraps、threading.Thread(target=..., args=..., kwargs=...)、测试框架的 subTest、一切装饰器——都在靠这对组合传递任意参数。

·初学者误区：f()（零参数）调用 def f(args) 得到空元组 ()，不是报错；(1,) 单元素元组必须有逗号；只能跟在之后（def f(kargs, a) 是 SyntaxError）。

18.11.2 调用中：解包参数 (Calls: Unpacking arguments)

英文原文

It turns out that we can use the syntax when we call a function, too. In this context, its meaning is the inverse of its meaning in the function definition—it unpacks a collection of arguments, rather than building a collection of arguments. For example, we can pass four arguments to a function in a tuple or other iterable, and let Python unpack them into individual positional arguments:

```
python >> def func(a, b, c, d): print(a, b, c, d) >> args = (1, 2) >> args += (3, 4) >> func(args) # Same as func(1, 2, 3, 4) 1 2 3 4
```

Similarly, the syntax in a function call unpacks a dictionary or other mapping of key/value pairs into separate keyword arguments:

```
python >> args = {'a': 1, 'b': 2, 'c': 3} >> args['d'] = 4 >> func(args) # Same as func(a=1, b=2, c=3, d=4) 1 2 3 4
```

Moreover, we can use other iterables like lists and other dictionary syntax like dict, and we may combine s

tar, positional, and keyword arguments in the call in very flexible ways:

```
python >> func((1, 2), {'d': 4, 'c': 3}) # Same as func(1,
2, d=4, c=3) 1 2 3 4 >> func(1, [2, 3], dict(d=4)) # Sa
me as func(1, 2, 3, d=4) 1 2 3 4 >> func(1, c=3, [2], {'
d': 4}) # Same as func(1, 2, c=3, d=4) 1 2 3 4 >> func(
1, (2, 3), d=4) # Same as func(1, 2, 3, d=4) 1 2 3 4 >>
func(1, [2], c=3, dict(d=4)) # Same as func(1, 2, c=3, d
=4) 1 2 3 4
```

While the preceding serves to demo the possibilities, its use of stars and literals is overkill when arguments are known; more typical code would build up argument collections ahead of the call and unpack by names:

```
python >> pargs, kargs = (1, 2), dict(d=4, c=3) >> fu
nc(pargs, kargs) 1 2 3 4
```

Star unpacking is convenient when you cannot predict the number of arguments that will be passed to a function when you write your script; you can build up a collection of arguments at runtime instead and call the function generically this way.

Here again, though, don't confuse the / starred-argument syntax in the function definition and the function call—in the definition it collects any number of arguments, while in the call it unpacks any number of arguments. In both, one star means positionals, and two applies to keywords.

Finally, as of Python 3.5, and as noted in Chapter 11's sidebar "The Many Stars of Python", we can even use multiple `*` and `**` in calls to unpack multiple iterables and mappings, respectively. The `*` unpacks into positional arguments, and into keyword arguments, though single stars must precede double stars, per formal rules coming up ahead. Again, the following uses literals to demo, but these would usually be names assigned to prebuilt values:

```
python >> def func(a, b, c, d): print(a, b, c, d) >> func([1], (2,), dict(c=3), {'d': 4}) # Positionals + keywords
1 2 3 4 >> func([1], (2,), [3, 4]) # All positionals
1 2 3 4 >> func(dict(a=1, b=2), dict(c=3), {'d': 4}) # All keywords
1 2 3 4 >> func([1], 2, dict(c=3), d=4) # All call modes at once!
1 2 3 4 >> func([1], dict(b=2, c=3), [4])
SyntaxError: iterable argument unpacking follows keyword argument unpacking
```

NOTE Unpacking generality—and inconsistency: As previewed in Chapter 14, the `*` form in a call is an iteration tool, so it accepts any iterable object, not just tuples or other sequences used in examples here. For instance, iterables like `zip` and `range` work after a `*` and unpack into individual arguments, and file objects automatically read and unpack their lines:

```
python func(range(4)) # Same as func(0, 1, 2, 3)
func(open('filename')) # Read+pass lines as arguments
```

Watch for more examples of this utility in Chapter 20, after we study generators. On the downside, stars also come with inconsistencies. For one, this generality holds true only for calls—a unpacking in a call allows any iterable, but a in a function definition always collects extra arguments into a tuple.

Moreover, this collection behavior in definitions is similar in spirit and syntax to the in the extended-unpacking assignment forms we met in Chapter 11 (e.g., `x, y = z`), but that star usage always creates lists, not tuples. Again: these are different rules for different tools—despite the same syntax.

中文翻译

事实证明，我们在调用函数时也可以使用语法。在这个上下文中，它的含义与函数定义中的含义正好相反——它解包

(unpacks) 一个参数集合，而不是构建一个参数集合。例如，我们可以把一个元组或其他可迭代对象中的四个参数传给函数，让 Python 把它们解包成独立的位置参数：`python >>> def func(a, b, c, d): print(a, b, c, d) >>> args = (1, 2) >>> args += (3, 4) >>> func(args) # 等价于 func(1, 2, 3, 4) 1 2 3 4`类似地，函数调用中的语法把一个字典或其他键/值对映射解包成独立的键字参数：`python >>> args = {'a': 1, 'b': 2, 'c': 3} >>> args['d'] = 4 >>> func(args) # 等价于 func(a=1, b=2, c=3, d=4) 1 2 3 4`此外，我们可以使用列表等其他可迭代对象、dict 等其他字典构造语法，还可以在调用中以极其灵活的方式组合星号、位置和键字参数：`python >>> func((1, 2), {'d': 4, 'c': 3})`

等价于 func(1, 2, d=4, c=3) 1 2 3 4 >>> func(1, [2, 3], dict(d=4)) # 等价于 func(1, 2, 3, d=4) 1 2 3 4 >>> func(1, c=3, [2], {'d': 4}) # 等价于 func(1, 2, c=3, d=4) 1 2 3 4 >>> func(1, (2, 3), d=4) # 等价于 func(1, 2, 3, d=4) 1 2 3 4 >>> func(1, [2], c=3, dict(d=4)) # 等价于 func(1, 2, c=3, d=4) 1 2 3 4 虽然前面的代码用来演示可能性，但当参数已知时，这种星号加字面量的写法就杀鸡用牛刀了；更典型的做法是：在调用前构建好参数集合，再按名字解包：python >>> pargs, kargs = (1, 2), dict(d=4, c=3) >>> func(pargs, kargs) 1 2 3 4 当你写脚本时无法预测会传给函数多少个参数时，星号解包就很方便；你可以改在运行时构建参数集合，然后用这种方式泛化地调用函数。不过这里再次提醒：不要把函数定义与函数调用中的 / 星号参数语法搞混——在定义中它收集任意数量的参数，而在调用中它解包任意数量的参数。两者中，一个星号都表示位置参数，两个星号都应用于关键字。最后，从 Python 3.5 起（正如第 11 章侧边栏“Python 的多星传奇”所提到的），我们甚至可以在调用中使用多个和项，分别解包多个可迭代对象和映射。解包成位置参数，解包成关键字参数，不过按照后面将要讲的正式规则，单星必须位于双星之前。同样，下面用字面量演示，但通常这些会是赋值给预构建值的名字：python >>> def func(a, b, c, d): print(a, b, c, d) >>> func([1], (2,), dict(c=3), {'d': 4}) # 位置参数 + 关键字 1 2 3 4 >>> func([1], (2,), [3, 4]) # 全是位置参数 1 2 3 4 >>> func(dict(a=1, b=2), dict(c=3), {'d': 4}) # 全是关键字 1 2 3 4 >>> func([1], 2, dict(c=3), d=4) # 一次用上所有调用模式！ 1 2 3 4 >>> func([1], dict(b=2, c=3), [4]) SyntaxError: iterable argument unpacking follows k

keyword argument unpacking

注意 (NOTE) 解包的通用性——以及不一致性：正如第 14 章预告的，调用中的形式是一种迭代工具，所以它接受任何可迭代对象，不只是这里示例中的元组或其他序列。例如，zip、range 这样的可迭代对象跟在后面也能工作并解包成独立参数，文件对象会自动读取并解包它们的行：`python func(range(4))` # 等价于 `func(0, 1, 2, 3)` `func(open('filename'))` # 读取文件并将每行作为参数传入更多这种实用性的例子，等我们学了生成器之后在第 20 章再看。不利的一面是，星号也伴随着不一致。其一，这种通用性只对调用成立——调用中的解包允许任何可迭代对象，但函数定义中的总是把多余参数收集成元组。其二，定义中的这种收集行为，在精神和语法上都与第 11 章学过的扩展解包赋值形式中的相似（如 `x, y = z`），但那种星号用法总是创建列表，而不是元组。再次强调：同一语法，不同工具的规则各不相同。

深度理解

- 核心概念：调用侧 / 与定义侧互为逆运算；Python 3.5+ 允许调用中多个与多个混排，但顺序受限（单星必须在双星前）。
- 底层实现：`func(args)` 编译为 `BUILDTUPLEUNPACK`（把多个可迭代对象展开合并进一个参数元组）；`func(kwargs)` 编译为 `BUILDMAPUNPACK`。`func(open('filename'))` 会一次性读取整个文件并逐行解包——因此对大文件要小心（迭代器被整体耗尽）。
- 为什么这样设计：泛化调度（generic dispatch）是核心

动机——写代码时不知道参数个数，运行时拼装参数集合即可。PEP 448 放宽为多个星号是为了对称性与拼接便利（如 `[a, b]`）。

·不一致性提醒：在“调用解包”（任意可迭代）、“定义收集”（元组）、“扩展解包赋值”（列表）三处行为不同——Lutz 特意强调“不同工具、不同规则、同一语法”，这是考试与面试经典考点。

·实际问题：`print(items)`、`dict(config)`、`subprocess.run(args, kwargs)`、装饰器转发、`super().init(args, kwargs)`——处处可见。

·初学者误区：在调用里对字典用（解出来的是键而不是值）；把解包结果传给没有对应形参的函数导致 `TypeError: unexpected keyword argument`；忘记 `range(4)` 是“展开后依次传”，而不是“传一个 `range` 对象”。

18.11.3 为什么需要任意参数？（Why arbitrary arguments?）

英文原文

The prior section's examples may seem academic (if not downright esoteric), but they are used more often than you might expect. Some programs need to call arbitrary functions in a generic fashion, without hardcoding their names or arguments ahead of time. In fact, the real power of the special starred-unpacking call syntax is that you don't need to know how many arguments a function call requires before you write a s

cript. For example, you can use if logic to select from a set of functions and argument lists, and call any of them generically (functions here are hypothetical):

```
python if sometest: action, args = func1, (1,) # Call func1 with one arg in this case
```

```
else: action, args = func2, (1, 2, 3) # Call func2 with three args instead
```

```
... action(args) # Dispatch generically
```

This leverages both the form and the fact that functions are objects that may be both referenced by, and called through, any variable. More generally, this unpacking call syntax is useful anytime you cannot predict the arguments list. If your user selects an arbitrary function via a user interface, for instance, you may be unable to hardcode a function call when writing your script. To work around this, simply build up the arguments list with sequence operations, and call it with starred-argument syntax to unpack the arguments:

```
python args = (2,3) args += (4,)
```

```
... func3(args)
```

Because the arguments list is passed in as a tuple here, the program can build it at runtime. This technique also comes in handy for functions that test or time other functions. For instance, the code in Example 18-1 supports any function with any arguments by passing along whatever arguments were sent in (see trace

r0.py in the example package).

Example 18-1. tracer0.py

```
python def tracer(func, pargs, kargs): # Accept arbitrary arguments
print('calling:', func.name) return func(pargs, kargs) # Pass along arbitrary arguments
def func(a, b, c, d): return a + b + c + d
print(tracer(func, 1, 2, c=3, d=4))
```

This code uses the built-in name attribute attached to every function (as you might expect, it's the function's name string), and uses stars to collect and then unpack the arguments intended for the traced function. In other words, when this code is run, arguments are intercepted by the tracer and then propagated with unpacking call syntax:

```
text $ python3 tracer0.py calling: func 10
```

For another example of this technique, see the preview near the end of the preceding chapter, where it was used to reset the built-in open function (though it probably makes more sense to you now). We'll code additional examples of such roles later in this book; see especially the sequence timing examples in Chapter 21 and the various decorator utilities we will code in Chapter 39 (after a preview in the next chapter). It's a common technique in general tools.

中文翻译

上一节的例子可能看起来像学术练习（甚至有点深奥），但它们的使用频率比你想象的高。有些程序需要以泛化的方式

调用任意函数，而不预先硬编码函数名或参数。事实上，特殊星号解包调用语法的真正威力在于：你不需要在写脚本之前就知道一次函数调用需要多少个参数。例如，你可以用 if 逻辑在一组函数和参数列表中做选择，并泛化地调用其中任何一个（这里的函数是假想的）：`python if sometest: action, args = func1, (1,) # 这种情况下用一个参数调用 func1 else: action, args = func2, (1, 2, 3) # 否则用三个参数调用 func2 ... action(args) # 泛化分发这同时利用了形式和"函数是对象"的事实——函数可以被任何变量引用、也可以通过任何变量调用。更一般地说，任何时候你无法预测参数列表，这种解包调用语法都有用。例如，如果你的用户通过用户界面选择了一个任意函数，你在写脚本时可能无法硬编码函数调用。变通办法很简单：用序列操作构建参数列表，然后用星号参数语法解包来调用它：python args = (2,3) args += (4,) ... func3(args) 因为这里的参数列表以元组传入，程序可以在运行时构建它。这个技巧对"测试或计时其他函数"的函数也很有用。例如，示例 18-1 中的代码通过原样转发收到的任何参数，支持任意函数与任意参数（参见示例包中的 tracer0.py）。示例 18-1. tracer0.py`

```
python def tracer(func, pargs, kargs): # 接受任意参数
print('calling:', func.name)
return func(pargs, kargs) # 原样转发任意参数
def func(a, b, c, d): return a + b + c + d
print(tracer(func, 1, 2, c=3, d=4))
```

这段代码使用了附加在每个函数上的内置 `name` 属性（如你所料，它是函数名的字符串），并用星号先收集、再解包本应传给被追踪函数的参数。换句话说，当这段代码运行时，参数被 `tracer` 截获，然后用解包调用语法转发出去：`text $ python3 tracer0.py calling: func 10` 这个技巧的另一个例子，参见上一章

末尾的预览——那里用同样的手法重置了内置 open 函数（不过现在你可能理解得更清楚了）。本书后面还会编写更多这类角色的例子；尤其参见第 21 章的序列计时示例，以及第 39 章我们将编写的各种装饰器工具（下一章有预览）。这是通用工具中的常见技术。

深度理解

- 核心概念："收集→转发" (collect-and-forward) 模式：
`def tracer(func, pargs, kargs): ... return func(pargs, kargs)`——参数进来什么形态，出去还是什么形态。这是装饰器、代理、日志、计时器的原型。
- 底层实现：`tracer(func, 1, 2, c=3, d=4)` 中，`pargs=(1, 2)`、`kargs={'c': 3, 'd': 4}`；`return func(pargs, kargs)` 时 `BUILDTUPLEUNPACK/BUILDMAPUNPACK` 把它们重新展开，恰好与 `func(1, 2, c=3, d=4)` 等价——参数经过"压缩-展开"往返，信息零丢失。
- 为什么这样设计：Python 把函数当一等对象 (first-class objects)，于是"函数套函数" (高阶函数) 成为可能；/ 就是高阶函数之间传递参数的"万能转接口"。
- 实际问题：`functools.wraps`、`unittest.mock`、Web 框架路由、`signal.signal`、以及第 39 章的装饰器——全是这个模式。第 21 章的计时器 (Timer) 案例是它的直接应用。
- 生活类比：前台总机 (tracer) 不关心你是谁、要打给谁——接通线路 (转发参数) 就算完成任务。总机只需要"原样接通"，不需要知道电话内容。

·初学者误区：在 `return func(pargs, kargs)` 里漏掉星号——那会把元组和字典整个当单个参数传给 `func`，得到 `TypeError`；写时混淆“定义收集”与“调用解包”的方向。

18.12 仅关键字参数 (Keyword-Only Arguments)

英文原文

Besides accepting keyword (i.e., pass-by-name) arguments in general, function definitions can also specify that some arguments must always be passed by keyword only and will never be filled in by a positional argument. This extension, known as keyword-only arguments, can be useful if we want a function to both process any number of arguments and accept possibly optional configuration options.

Syntactically, keyword-only arguments are coded as named arguments that may appear after name in the arguments list. All such arguments must be passed using keyword syntax in the call. For example, in the following, `a` may be passed by name or position, `b` collects any extra positional arguments, and `c` must be passed by keyword only:

```
python >> def konly(a, b, c): print(a, b, c) >> konly(1, 2, c=3) 1 (2,) 3 >> konly(a=1, c=3) 1 () 3 >> konly(1, 2, 3) TypeError: konly() missing 1 required k
```

keyword-only argument: 'c'

If we don't need to collect arbitrary positionals, we can also use a bare character by itself in the argument list to introduce keyword-only arguments. This indicates that a function does not accept a variable-length argument list but still expects all arguments following the to be passed as keywords. In the next function, `a` may be passed by position or name again, but `b` and `c` must be keywords, and no extra positionals are allowed:

```
python >> def kwnonly(a, , b, c): print(a, b, c) >> kwnonly(1, c=3, b=2) 1 2 3 >> kwnonly(c=3, b=2, a=1) 1 2 3 >> kwnonly(1, 2, 3) TypeError: kwnonly() takes 1 positional argument but 3 were given >> kwnonly(1) TypeError: kwnonly() missing 2 required keyword-only arguments: 'b' and 'c'
```

You can still use defaults for keyword-only arguments, even though they appear after the in the function. In the following, `a` may be passed by name or position, and `b` and `c` are optional but must be passed by keyword if used:

```
python >> def kwnonly(a, , b='code', c='app'): print(a, b, c) >> kwnonly(1) 1 code app >> kwnonly(1, c='hack') 1 code hack >> kwnonly(a='py') py code app >> kwnonly(c=3, b=2, a=1) 1 2 3 >> kwnonly(1, 2) TypeError: kwnonly() takes 1 positional argument but 2 were given
```

In fact, keyword-only arguments with defaults are optional, but those without defaults effectively become required keywords for the function—like `b` in the following:

```
python >> def konly(a, , b, c='hack'): print(a, b, c) >
> konly(1, b='code') 1 code hack >> konly(1, c='code')
TypeError: konly() missing 1 required keyword-only argument:'b'>> konly(1, 2)
TypeError: konly() takes 1 positional argument but 2 were given
```

As noted earlier, keyword-only arguments also allow defaults and nondefaults to be mixed, unlike their otherwise more flexible cohorts coded before the optional—an ostensible inconsistency we'll return to later:

```
python >> def konly(a, , b=2, c, d=4): print(a, b, c, d)
>> konly(1, c=3) 1 2 3 4 >> konly(1, c=5, b=6) 1 6 5 4
>> konly(1)
TypeError: konly() missing 1 required keyword-only argument:'c'>> konly(1, 2, 3)
TypeError: konly() takes 1 positional argument but 3 were given
>> def badmix(b=2, c, d=5):...
SyntaxError: parameter without a default follows parameter with a default
```

Finally, note that keyword-only arguments must be specified after a single star, not two—nothing can appear after the `args` arbitrary-keywords form, and, unlike `a`, `a` can't appear by itself in the arguments list. More generally, keyword-only arguments must be coded between the `and` and the optional `,` and an argument that

appears before is a possibly default argument that can be passed by position or keyword, not keyword-only:

```
python >> def kwonly(a, kargs, b, c): SyntaxError: arguments cannot follow var-keyword argument >> def kwonly(a, , b, c): SyntaxError: invalid syntax >> def mixed(a, b, d, c=6): SyntaxError: arguments cannot follow var-keyword argument
```

These failures will make more sense after we get formal about argument ordering rules later in this chapter. They may also appear to be worst cases in the artificial examples here, but they can come up in real practice, especially for people who write libraries for other Python programmers to use—which leads to the next point.

中文翻译

除了普通地接受关键字（即按名字传递）参数之外，函数定义还可以指定某些参数必须仅以关键字方式传递，绝不能被位置参数填充。这个扩展被称为仅关键字参数（keyword-only arguments），当我们希望一个函数既能处理任意数量的参数、又能接受可能可选配置选项时，它非常有用。从语法上说，仅关键字参数被写成出现在参数列表 name 之后的有名字的参数。所有这类参数在调用中都必须使用关键字语法传递。例如，在下面的代码中，a 可以按名字或位置传递，b 收集任何多余位置参数，而 c 必须仅以关键字传递：

```
python >>> def kwonly(a, b, c): print(a, b, c) >>> kwonly(1, 2, c=3) 1 (2,) 3 >>> kwonly(a=1, c=3) 1 () 3
```

>>> kwonly(1, 2, 3) TypeError: kwonly() missing 1 required keyword-only argument:'c'如果我们不需要收集任意位置参数，也可以在参数列表中单独写一个裸字符来表示仅关键字参数的开始。这表示该函数不接受变长位置参数列表，但仍然期望之后的所有参数以关键字方式传递。在下一个函数中，a 同样可以按位置或名字传递，但 b 和 c 必须是关键字，而且不允许多余的位置参数：python >>> def kwonly(a, , b, c): print(a, b, c) >>> kwonly(1, c=3, b=2) 1 2 3 >>> kwonly(c=3, b=2, a=1) 1 2 3 >>> kwonly(1, 2, 3) TypeError: kwonly() takes 1 positional argument but 3 were given >>> kwonly(1) TypeError: kwonly() missing 2 required keyword-only arguments:'b' and 'c'你仍然可以为仅关键字参数使用默认值，即使它们出现在函数的之后。在下面的代码中，a 可以按名字或位置传递，b 和 c 是可选的，但如果使用就必须以关键字传递：python >>> def kwonly(a, , b='code', c='app'): print(a, b, c) >>> kwonly(1) 1 code app >>> kwonly(1, c='hack') 1 code hack >>> kwonly(a='py') py code app >>> kwonly(c=3, b=2, a=1) 1 2 3 >>> kwonly(1, 2) TypeError: kwonly() takes 1 positional argument but 2 were given 事实上，带默认值的仅关键字参数是可选的，但不带默认值的仅关键字参数实际上就变成函数的必需关键字（required keywords）——就像下面代码中的 b：python >>> def kwonly(a, , b, c='hack'): print(a, b, c) >>> kwonly(1, b='code') 1 code hack >>> kwonly(1, c='code') TypeError: kwonly() missing 1 required keyword-only argument:'b' >>> kwonly(1, 2) TypeError: kwonly() takes 1 positional argument but 2 were given 如前所述

，仅关键字参数还允许默认值与非默认值混排——这与可选之前的那些（其他方面更灵活的）参数不同——这是一个表面上的不一致，我们后面会再回来讨论：

```
python >>> def kwonly(a, b=2, c, d=4): print(a, b, c, d) >>> kwonly(1, c=3) 1 2 3 4 >>> kwonly(1, c=5, b=6) 1 6 5 4 >>> kwonly(1) TypeError: kwonly() missing 1 required keyword-only argument:'c'>>> kwonly(1, 2, 3) TypeError: kwonly() takes 1 positional argument but 3 were given >>> def badmix(b=2, c, d=5): ... SyntaxError: parameter without a default follows parameter with a default
```

最后要注意：仅关键字参数必须位于单星之后，而不是双星——args 任意关键字形式后面不能出现任何东西，而且与不同，不能在参数列表中单独出现。更一般地说，仅关键字参数必须编码在与可选的之间；而出现在之前的参数，是一个可能带默认值、可以按位置或关键字传递的参数，不是仅关键字参数：

```
python >>> def kwonly(a, kargs, b, c): SyntaxError: arguments cannot follow var-keyword argument >>> def kwonly(a, , b, c): SyntaxError: invalid syntax >>> def mixed(a, b, d, c=6): SyntaxError: arguments cannot follow var-keyword argument
```

这些失败的原因，等我们本章后面正式讨论参数排序规则时会更清楚。它们在这里的人工示例中看起来像是极端情况，但在实际实践中可能出现，尤其是对于那些为其他 Python 程序员写库的人——这就引出下一点。

深度理解

·核心概念：仅关键字参数 = 必须按名传递的参数。它有两种声明方式：放在 name 之后（顺便还能收集位置参数

)，或放在裸之后（不收集位置参数）。

·底层实现：函数对象用 `code.coonlyargcount` 记录仅关键字参数的数量；匹配时，这些参数的槽位不会被任何位置参数填充——位置参数分配阶段直接跳过它们（因此不足时报 `missing N required keyword-only argument`）。

·为什么这样设计（PEP 3102）：让 `def process(args, notify=False)` 里的 `notify` 不会与前缀位置参数混淆——位置参数永远碰不到它，配置选项既可选又不会“误吃”参数。

·特殊规则：仅关键字参数允许默认值/非默认值混排（`def f(a, b=2, c, d=4)` 合法），因为它永远不按位置匹配，默认值不产生位置歧义；这对之前参数的“默认值必须连续”规则是个特例。

·实际问题：标准库中 `sorted(iterable, key=None, reverse=False)`、`print(objects, sep=" ", end="\n", ...)`、`max(iterable, default=...)` 都在用；新手自己写“配置项 + 任意位置参数”组合的函数时，这是教科书级解法。

·初学者误区：以为裸是“收集参数”——不，裸是“分界线”，只负责宣告其后的参数只能按关键字传；把仅关键字参数变成必需后漏传，会得到难懂的错误信息（对非英语用户尤其如此）。

18.12.1 为什么需要仅关键字参数? (Why key word-only arguments?)

英文原文

So why care about keyword-only arguments? In short, they make it easier to allow a function to accept both any number of positional arguments to be processed, and configuration options passed as keywords. While their use is optional, without keyword-only arguments extra work may be required to provide defaults for such options and to verify that no superfluous keywords were passed. Imagine a function that processes a set of passed-in objects and allows a tracing flag to be passed: `python process(X, Y, Z)` # Use flag's default `process(X, Y, notify=True)` # Override flag default Without keyword-only arguments we have to use both `args` and `kwargs` and manually inspect the keywords, but with keyword-only arguments less code is required. The following guarantees that no positional argument will be incorrectly matched against `notify` and requires that it be a keyword if passed: `python def process(args, notify=False): ...` Since we're going to explore a more realistic example of this later in this chapter, in "Example: Rolling Your Own Print", we'll postpone the rest of this story until then. For an additional example of keyword-only arguments in action, see the upcoming iteration-options timing case study in

中文翻译

那么为什么要在意仅关键字参数呢？简言之，它们让一个函数更容易同时做到：接受任意数量的位置参数供处理，以及接受以关键字方式传入的配置选项。虽然它们的使用是可选的，但没有仅关键字参数，你可能需要做额外工作来为这些选项提供默认值，并验证没有传入多余的关键字。设想一个函数：处理一组传入对象，并允许传入一个追踪（tracing）标志：
`python process(X, Y, Z) # 使用标志的默认值`
`process(X, Y, notify=True) # 覆盖标志的默认值`
没有仅关键字参数，我们不得不同时使用 `args` 和 `kwargs`，并手动检查关键字；而有了仅关键字参数，需要的代码更少。下面的函数保证不会有位置参数被错误地匹配到 `notify`，而且如果传了就必须是关键字：
`python def process(kwargs, notify=False):`
... 因为本章稍后在"示例：自己动手写 `print`"中会探索这个更现实的例子，我们把这个故事的其余部分推迟到那时。仅关键字参数实战的另一个例子，参见即将在"计时器模块：第二版"（Timer Module: Take 2）中出现的迭代选项计时案例研究。

深度理解

- 核心概念：仅关键字参数把"配置选项"从"数据参数"中物理隔离出来——位置参数只能撞上数据，永远撞不上配置项。
- 对比：方案 A `def process(args, kwargs) + 手动 kwargs.get('notify', False) + 手动检查多余键`（还要冒着拼错键名

被静默忽略的风险)；方案 B `def process(args, notify=False)`——一行语法，自动校验，错误键名直接 `TypeError`。这是 18.16.1 节 `print3` 三个版本的进化主线。

·为什么这样设计：错误定位——如果 `notify` 是普通参数，`process(X, Y, Z, True)` 里的 `True` 究竟给谁？仅关键字让“给配置项”必须点名，杜绝这类歧义。

·生活类比：食堂窗口点餐——`args` 是一串菜（无论多少盘），`notify` 是“打包/加辣”这种要求。要求必须单独说（关键字），绝不能让排队插进来的菜顶替“是否打包”。

·初学者误区：认为 `kwargs` 够用所以不需要仅关键字——但 `kwargs` 不会自动报出拼错的配置键，调试浪费时间；仅在“真正需要任意未定义选项”时才用 `kwargs`。

18.13 仅位置参数 (Positional-Only Arguments)

英文原文

Beginning with Python 3.8, function definitions may also include a `/` in the arguments list to designate that all arguments preceding it (i.e., to its left) must be passed by position, not by keyword-argument name.

Though arguably ad hoc on first sighting in an arguments list, this notation was being used in documentation for built-in functions that did not accept keywords; making it available to function coders as part of Py

thon's syntax was deemed a logical extension for library developers who may not want to expose argument names for use by clients as keywords.

To demo, the following function specifies that `a` and `b` must be passed by position, though `c` is more flexible:

```
python >> def mostlypos(a, b, /, c): print(a, b, c) >>
mostlypos(1, 2, 3) 1 2 3 >> mostlypos(1, 2, c=3) 1 2 3
```

Passing either of the first two arguments by keyword, however, fails:

```
python >> mostlypos(1, b=2, c=3) TypeError: mostly
pos() got some positional-only arguments passed as
keyword arguments:'b'>> mostlypos(c=3, b=2, a=1)
TypeError: mostlypos() got some positional-only argu
ments passed as keyword arguments:'a, b'
```

To define a function that allows only positional arguments, simply code the slash at the end:

```
python >> def allpos(a, b, c, /): print(a, b, c) >> allpo
s(1, 2, 3) 1 2 3 >> allpos(1, 2, c=3) TypeError: mostlyp
os() got some positional-only arguments passed as k
eyword...
```

As you should expect, the slash works the same in a lambda argument list:

```
python >> f = lambda a, b, /, c: print(a, b, c) >> f(1, 2
, c=3) 1 2 3 >> f(1, b=2, c=3) TypeError: <lambda>()
got some positional-only arguments passed as keyw
```

ord...

And functions can combine positional- and keyword-only arguments to be as rigid as they wish:

```
python >> def combo(a, b, /, , c, d): print(a, b, c, d) >  
> combo(1, 2, c=3, d=4) 1 2 3 4 >> combo(1, 2, 3, 4)  
TypeError: combo() takes 2 positional arguments but  
4 were given >> combo(a=1, b=2, c=3, d=4) TypeErr  
or: combo() got some positional-only arguments pas  
sed as keyword...
```

It's up to you to ponder whether or not use cases for this syntax justify its convolution of function definitions that follows in the next section. Given that Python users somehow got by without it for over three decades, though, this seems a tough sell. For more on the rationale and usage of the positional-only slash, see Python's standard manuals. Also watch for an example in Chapter 21 that uses it to avoid name clashes—and may or may not be compelling.

中文翻译

从 Python 3.8 开始，函数定义还可以在参数列表中包含一个 /，用来指定它前面（即它左边）的所有参数必须按位置传递，而不能用关键字参数名传递。虽然这个记号乍一看有点“临时起意”（ad hoc），但它一直用于那些不接受关键字的内置函数的文档中；把它作为 Python 语法的一部分提供给函数作者，被认为是对库开发者的一个合理扩展——他们可能不想把参数名暴露给使用方当关键字来用。为了演

示，下面的函数指定 a 和 b 必须按位置传递，而 c 更灵活：

```
python >>> def mostlypos(a, b, /, c): print(a, b, c)
>>> mostlypos(1, 2, 3) 1 2 3 >>> mostlypos(1, 2, c=3) 1 2 3
```

但前两个参数中任何一个以关键字方式传递都会失败：

```
python >>> mostlypos(1, b=2, c=3) TypeError: mostlypos() got some positional-only arguments passed as keyword arguments:'b'
>>> mostlypos(c=3, b=2, a=1) TypeError: mostlypos() got some positional-only arguments passed as keyword arguments:'a, b'
```

要定义一个只允许位置参数的函数，把斜杠写在末尾即可：

```
python >>> def allpos(a, b, c, /): print(a, b, c)
>>> allpos(1, 2, 3) 1 2 3 >>> allpos(1, 2, c=3) TypeError: mostlypos() got some positional-only arguments passed as keyword ...
```

正如你所料，斜杠在 lambda 参数列表中的工作方式相同：

```
python >>> f = lambda a, b, /, c: print(a, b, c)
>>> f(1, 2, c=3) 1 2 3 >>> f(1, b=2, c=3) TypeError: <lambda>() got some positional-only arguments passed as keyword ...
```

而且函数可以把仅位置与仅关键字参数组合起来，想多严格就多严格：

```
python >>> def combo(a, b, /, c, d): print(a, b, c, d)
>>> combo(1, 2, c=3, d=4) 1 2 3 4 >>> combo(1, 2, 3, 4) TypeError: combo() takes 2 positional arguments but 4 were given
>>> combo(a=1, b=2, c=3, d=4) TypeError: combo() got some positional-only arguments passed as keyword ...
```

是否认可这种语法足以付出下一节将要出现的、函数定义的复杂化代价，就由你权衡了。不过，考虑到 Python 用户三十多年来没有它也能活得挺好，这似乎有点难推销。关于仅位置斜杠的理据与用法，更多内容参见 Python 标准手册。

另外请注意第 21 章有一个用它来避免名字冲突的例子——其说服力嘛，见仁见智。

深度理解

·核心概念：/ 是"仅位置"分界线——它左边必须按位置传，右边按需。combo(a, b, /, , c, d) 是"严格模式"的极致：中间夹着声明仅关键字，两侧分别是仅位置与仅关键字。

·底层实现：函数对象用 code.coonlyargcount 记录仅位置参数数量。调用时，若调用方用关键字名指到这些槽位，运行期抛 TypeError: got some positional-only arguments passed as keyword arguments。这也是为什么 pow(x, y, /, mod=None)、sorted(iterable, /, , key=None, reverse=False) 打印 help 时能看到 /。

·为什么这样设计 (PEP 570)：让内置函数能从"不接受关键字"固化为语法——更重要的是，给了库作者一个自由：参数名是私有实现细节，将来改名不破坏调用方的关键字用法。

·实际问题：math 模块大量使用 (math.log(x, /, base=None))；希望将来自由改参数名的库函数；以及第 21 章那个避免名字冲突的例子。

·Lutz 的态度：作者直言这是"难推销"的语法——三十多年没有它也能过。这提醒读者：新语法好用，但不必为用而用。

·初学者误区：把 / 误写成 \；以为 / 之后的参数"必须用关键字"——不，/ 只约束其前的参数；混淆 / 与的职责（

一个管"左边只能位置", 一个管"右边只能关键字")。

·生活类比: 取号排队——/ 前的参数像"必须到柜台取号" (不能网上预约), 后的参数像"必须实名预约" (不能现场插队)。两者的限制方向正好相反。

18.14 参数排序: 繁琐细节 (Argument Ordering: The Gritty Details)

英文原文

So far, we've been fairly loose about the rules surrounding argument-matching tools, because they don't crop up very often in the simpler usage patterns of typical code. In more sophisticated roles, though, you need to verify that your code follows Python's expectations. Now that we've seen all of its subjects, the ordering rules for function arguments can finally be summarized in full. While function definitions and calls share some similar syntax, their rules are completely different, owing to their different roles. Let's take a more formal look at both.

中文翻译

到目前为止, 我们对参数匹配工具周围的规则一直比较宽松, 因为在典型代码的简单用法模式中, 它们很少冒出来。但在更复杂的角色中, 你需要确保代码符合 Python 的预期。现在我们已经见过全部"角色", 函数参数的排序规则终于可

以完整汇总了。虽然函数定义与调用共享一些相似的语法，但由于两者角色不同，规则完全不同。让我们对两者做一次更正式的审视。

深度理解

·核心概念：排序规则是“语法无歧义”的保险。定义侧与调用侧语法相似（都有星号、有 =），但语义相反、规则独立——这是本章最容易“串台”的地方。

·为什么这样设计：Python 编译器必须在编译期就能唯一解析参数的归属；任何允许歧义的排列都会直接 SyntaxError，把错误提前到编译期而非运行期。

18.14.1 定义中的排序 (Definition Ordering)

英文原文

In function definitions, argument lists are enclosed in parentheses in a def statement and coded before a colon in a lambda expression, but follow the same format in both. In short, they consist of four optional parts that must appear in the following order, where "position" means a simple value in calls, and "keyword" means a name=value pair: 1. One or more arguments that must be passed by position only, followed by a single / 2. Any number of arguments that can be passed by either position or keyword 3. A single by itself or a single name positional-argument collector, optionally followed by any number of arguments that must be passed by keyword only 4. A single name keywor

d-argument collector In all cases, individual arguments, including a bare / or , are separated by commas. In addition, any nonstarred argument name can have a name=expression default, but all names must have defaults after the first that does, up to the (keyword-only argument runs following a star may freely mix default and nondefault names). Inherent in this ordering, positional-only arguments must appear first, name ends positional-argument runs and collects unmatched positionals, and name ends the entire arguments list and collects unmatched keywords. As noted earlier, the name collector's keys retain the order in which keyword arguments were passed to the function. Formal definition. More concisely, the ordering of arguments in function definitions can be defined as follows, where -or- is notation for a choice and [] encloses an optional part (neither is part of the actual code you type):

```
text def name(arguments-list): statements
lambda arguments-list: expression
arguments-list = [positional-only-arguments, /] [positional-or-keyword-arguments] [ -or- positional-collector, [keyword-only-arguments]] [keyword-collector]
```

As a real example, the following defines a function with all these parts in actions:

```
python >>> def f(a, /, b, c=3, ps, e=4, ks): ... print(f'{a=}, {b=}, {c=}, {ps=}, {e=}, {ks=}') >>> f(0, 1, 2, 3, 4, e
```

=5, f=6, g=7) a=0, b=1, c=2, ps=(3, 4), e=5, ks={'f': 6, 'g': 7} Boundary cases. As consequences of the ordering in function definitions, / must precede a star, and the double-star keyword collector must be coded last :

```
python >>> def f(a, ps, /, b): pass
SyntaxError: / must be ahead of
```

```
>>> def f(a, ks, b): pass
SyntaxError: arguments cannot follow var-keyword argument
```

Though enforced separately from basic argument-ordering rules, once a default is used in a definition, all subsequent arguments must also have defaults—including those following the positional-only / delimiter:

```
python >>> def f(a, b, c=3, d, e): pass
SyntaxError: parameter without a default follows parameter with a default
```

```
>>> def f(a, b=2, /, c): pass
SyntaxError: parameter without a default follows parameter with a default
```

However, this constraint applies only to arguments preceding a or name—defaults and nondefaults can be mixed freely in keyword-only arguments, which seems inconsistent, though a case for sanity could be made here on the grounds that keyword-only arguments never match by position, which reduces ambiguity of defaults:

```
python >>> def f(a, , x=1, y): ... # OK: x and y must be keywords
>>> def f(a=1, b, , x=1): ... # Not OK: a and b may be positional
SyntaxError: parameter without a default follows parameter with a default
```

Not shown here, `def` can also be preceded by a decorator (e.g., `@value`), and, as noted earlier, any of its arguments may be followed by annotations (e.g., `:value`)—extensions we'll explore in the next chapter and later in this book. Neither of these impact argument ordering or matching, and neither work in `lambda` due to its limited syntax.

中文翻译

在函数定义中，参数列表在 `def` 语句里放在括号中，在 `lambda` 表达式里写在大冒号之前，但两者遵循完全相同的格式。简言之，它们由四个可选部分构成，必须按以下顺序出现，其中“位置”指调用中的简单值，“关键字”指 `name=value` 对：

1. 一个或多个必须仅按位置传递的参数，后面跟单个 /
2. 任意数量可以按位置或关键字传递的参数
3. 单个裸或单个 `name` 位置参数收集器，其后可选地跟任意数量必须仅按关键字传递的参数
4. 单个 `name` 关键字参数收集器

在所有情况下，各个参数（包括裸 / 或）之间用逗号分隔。此外，任何非星号参数名都可以有 `name=expression` 默认值，但一旦某个参数使用了默认值，到为止的所有后续名字都必须有默认值（星号之后的仅关键字参数序列可以自由混排带默认值与不带默认值的名字）。这个排序天然意味着：仅位置参数必须最先出现，`name` 终结位置参数序列并收集未匹配的位置参数，`name` 终结整个参数列表并收集未匹

配的关键字参数。如前所述，name 收集器的键保留关键字参数传给函数的顺序。正式定义。更简洁地说，函数定义中参数的排序可以定义如下，其中 -or- 表示"二选一"，[] 括起可选部分（两者都不是你实际输入代码的一部分）：text

```
def name(arguments-list): statements
lambda arguments-list: expression
arguments-list = [positional-only-arguments, /] [positional-or-keyword-arguments] [ -or- positional-collector, [keyword-only-arguments]] [ keyword-collector]
```

举一个真实例子，下面的函数把所有部分都完美演示了一遍：

```
python >>> def f(a, /, b, c=3, ps, e=4, ks): ... print(f'{a=}, {b=}, {c=}, {ps=}, {e=}, {ks=}')
>>> f(0, 1, 2, 3, 4, e=5, f=6, g=7)
a=0, b=1, c=2, ps=(3, 4), e=5, ks={'f': 6, 'g': 7}
```

边界情况。作为定义排序的推论：/ 必须位于星号之前，双星关键字收集器必须写在最后：

```
python >>> def f(a, ps, /, b): pass
SyntaxError: / must be ahead of
>>> def f(a, ks, b): pass
SyntaxError: arguments cannot follow var-keyword argument
```

虽然有别于基本参数排序规则、被单独强制，但一旦定义中使用了默认值，其后所有参数也都必须有默认值——包括仅位置 / 分隔符之后的参数：

```
python >>> def f(a, b, c=3, d, e): pass
SyntaxError: parameter without a default follows parameter with a default
>>> def f(a, b=2, /, c): pass
SyntaxError: parameter without a default follows parameter with a default
```

然而，这个约束只适用于或 name 之前的参数——仅关键字参数中可以自由混排默认值与非默认值。这看起来不一致，但也可以为它的合理性辩护：仅关键字参数永远不会按位置匹配，这就降低了默认值的歧义：

```
python >>> def f(a, , x=1, y): ... # 合法：x 和
```

y 必须是关键字 >>> def f(a=1, b, , x=1): ... # 不合法: a 和 b 可能按位置传递 SyntaxError: parameter without a default follows parameter with a default 这里没有展示的是: def 前面还可以有装饰器 (decorator, 如 @value), 而且如前所述, 它的任何参数后面都可以跟注解 (annotation, 如 :value) ——这些扩展我们将在下一章及本书后面探索。这两者都不影响参数排序或匹配, 而且由于 lambda 语法受限, 它们都不能用于 lambda。

深度理解

·核心概念: 定义侧四段式签名图谱: (仅位置, /, 位置或关键字, 收集器, 仅关键字, 收集器)。完整签名实例 def f(a, /, b, c=3, ps, e=4, ks) 涵盖了全部语法。

·底层实现: coposonlyargcount、coargcount、cokwonlyargcount 三个计数 + COVARARGS/COVARKEYWORDS 两个标志位, 共同把签名完整描述给运行时匹配器。

·默认值约束的"不对称": 之前"一旦用默认值就全部要用默认值" (因为按位置匹配时必须无歧义); 之后可混排 (因为只按名匹配)。这是本章反复强调的"表面不一致、实则合理"。

·实用场景: def sort(seq, /, , key=None, reverse=False) 这类"首参数仅位置 + 配置仅关键字"的现代签名, 正是四段式图谱的标准应用。

·初学者误区: 把 / 与的相对顺序记反 (/ 必须在之前); 在 lambda 里写装饰器或注解 (语法不支持); 以为 def f(a=1, b, , x=1) 合法——前的 b 无默认值是 SyntaxError

r。

18.14.2 调用中的排序 (Calls Ordering)

英文原文

On the other side of the fence, the syntax is similar but the rules differ. In function calls, all positional arguments must precede all keyword arguments, and any number of starred unpackings can be mixed in with individual values: one star unpacks into multiple positional arguments, two unpacks into keywords, and all positional arguments and one-star unpackings must precede all two-star unpackings. In more detail, function (and by extension, method) calls consist of three optional parts in the following order: 1. Any combination of one or more expression positional arguments, and expression iterable unpackings transformed into positional arguments 2. Any combination of one or more name=expression keyword arguments, and expression iterable unpackings transformed into positional arguments 3. Any combination of one or more name=expression keyword arguments, and expression mapping unpackings transformed into keyword arguments In all cases, all arguments, starred or otherwise, are separated by commas. Formal definition. More concisely again, the ordering of arguments in function calls can be defined as follows, where function is an expression that evaluates to a callable object, and -or-

and [] here again mean a choice and optional part, respectively (they're not part of the code you type):

```
text function([positional-values -or- iterable-positional-unpackings] [keyword-arguments -or- iterable-positional-unpackings] [keyword-arguments -or- mapping-keyword-unpackings])
```

As a concrete example, the following passes a variety of positional, keyword, and unpacking arguments (this code serves as a demo here, but is not exactly the sort of thing you should strive to craft in practice!):

```
python >>> def f(a, b, c, d, e, f, g, h, i): ... print(a, b, c, d, e, f, g, h, i) >>> f([1], 2, [3], 4, f=6, [5], dict(g=7), h=8, {'i': 9}) 1 2 3 4 5 6 7 8 9
```

Boundary cases. As one consequence of the argument ordering in calls, once you use a keyword argument, you can no longer use any unstarred positionals—all subsequent arguments must also be keywords, or single or double stars:

```
python >>> f(1, 2, c=3, 4) SyntaxError: positional argument follows keyword argument unpacking
```

This is similar to defaults in definitions—but not exactly. Again, try not to conflate function definitions and calls. Despite their reuse of similar syntax, it has very different roles and rules in these tools. The `and`, `=`, `f` or example, are used for unpacking and keywords in calls, but mean collection and defaults in definitions.

Once you code a double star, both positionals and single stars are also out of the game, though this seems inconsistent too—single stars can be freely mixed with keyword arguments (which is what a double star unpacks into), and keyword arguments cannot be mixed with positionals (which is what a single star unpacks into):

```
python >>> f(1, 2, {'d': 3}, 4) SyntaxError: positional argument follows keyword argument unpacking
```

```
>>> f(1, 2, {'d': 3}, [4]) SyntaxError: iterable argument unpacking follows keyword argument unpacking
```

```
>>> f(1, 2, d=3, [4]) # OK, but why? - like first error above  
>>> f(1, 2, d=3, 4) # Not OK, but why? - like preceding line  
SyntaxError: positional argument follows keyword argument
```

中文翻译

在篱笆的另一边，语法相似但规则不同。在函数调用中，所有位置参数必须位于所有关键字参数之前，任意数量的星号解包可以与单个值混排：一个星号解包成多个位置参数，两个星号解包成关键字参数，所有位置参数和单星解包都必须位于所有双星解包之前。更详细地说，函数（进而包括方法）调用由三个可选部分组成，顺序如下：1. 任意组合的一个或多个 expression 位置参数，以及转换成一个或多个位置参数的 expression 可迭代解包2. 任意组合的一个或多个 name=expression 关键字参数，以及转换为位置参数的 e

xpression 可迭代解包3. 任意组合的一个或多个 name=expression 关键字参数，以及转换为关键字参数的 expression 映射解包在所有情况下，所有参数（带星号与否）之间都用逗号分隔。正式定义。更简洁地说，函数调用中参数的排序可以定义如下，其中 function 是求值结果为可调用对象的表达式，-or- 和 [] 同样分别表示"二选一"和"可选部分"（它们不是你输入代码的一部分）：text function([positional-values -or- iterable-positional-unpackings] [keyword-arguments -or- iterable-positional-unpackings] [keyword-arguments -or- mapping-keyword-unpackings]) 举一个具体例子，下面这个调用混用了各种位置、关键字和解包参数（这段代码在这里只是演示，并不是你实践中应该追求写出来的东西！）：python >>> def f(a, b, c, d, e, f, g, h, i): ... print(a, b, c, d, e, f, g, h, i) >>> f([1, 2, [3], 4, f=6, [5], dict(g=7), h=8, {'i': 9}) 1 2 3 4 5 6 7 8 9 边界情况。作为调用参数排序的一个推论：一旦你使用了一个关键字参数，就不能再使用任何不带星号的位置参数了——之后的所有参数也必须是关键字、单星或双星：python >>> f(1, 2, c=3, 4) SyntaxError: positional argument follows keyword argument unpacking 这与定义中的默认值规则相似——但又不完全一样。再次提醒：不要把函数定义与调用混为一谈。尽管它们复用了相似的语法，但在这两种工具中它有着截然不同的角色与规则。例如，和 = 在调用中用于解包和关键字，但在定义中表示收集和默认值。一旦你写了双星，位置参数和单星就都出局了——尽管这看起来也不一致：单星可以与关键字参数自由混排（双星解包出来的正是关键字参数），而关键字参数不能与位置参数混排（单星解包出来的正是位置参数）：python >>> f(

1, 2, {'d': 3}, 4) SyntaxError: positional argument follows keyword argument unpacking >>> f(1, 2, {'d': 3}, [4]) SyntaxError: iterable argument unpacking follows keyword argument unpacking >>> f(1, 2, d=3, [4]) # 合法, 但为什么? ——如同上面第一个错误>>> f(1, 2, d=3, 4) # 不合法, 但为什么? ——如同前一行SyntaxError: positional argument follows keyword argument

深度理解

·核心概念: 调用侧三段式: (位置值/单星) → (关键字/单星) → (关键字/双星)。单星属于"位置阵营", 双星属于"关键字阵营"。

·一个"看似矛盾"的妙点: f(1, 2, d=3, [4]) 合法, 而 f(1, 2, d=3, 4) 不合法——因为 [4] 是"单星解包", 它被解析为位置参数, 语法上属于第一阶段/第二阶段的组成部分 (规则 2 允许关键字与单星混排); 而裸 4 是"未经包装的位置值", 出现在关键字之后即违反规则 2。所以: 星号成了"位置值的通行证"——它的展开结果被视为序列展开操作, 而不是一个"位置值"。

·为什么这样设计: expr 与 expr 在字节码层面是 BUILD_TUPLE/UNPACK/BUILDMAP/UNPACK, 它们把"集合展开"作为一种整体操作, 可安全地与相邻语法混排; 而裸值需要明确的坑位。正因如此, 错误信息里两类报错的措辞不同 ("positional argument follows keyword argument" vs "...unpacking...")。

·生活类比: 装盒寄快递——裸值像单独一件货, 必须按顺

序（先位置箱后关键字箱）放入；星号像"整箱散货倒出来"，可以倒进合并区。"d=3 之后又来一个裸 4"，就像你已开始填"注明收件人"的关键字表格，又硬塞进来一件无名裸件——系统不知道该把它放哪一栏。

·初学者误区：凭感觉混排位置/关键字/星号导致 SyntaxError 却看不明白；把错误信息里的 unpacking 字样当成"装包问题"——其实多半是"顺序问题"。

18.14.3 视角 (Perspective)

英文原文

And if this is starting to make your head spin, it probably should. Python's argument-matching rules have been accumulated over time to incorporate new convolutions, and they are complex and perhaps even kludgy. The first rule of programming applies to function definitions and calls as everywhere else: keep it simple, unless it has to be complex. If you have to agonize over argument-ordering rules to understand code, it's probably time to reevaluate priorities.

中文翻译

如果这些开始让你头晕目眩，那大概是应该的。Python 的参数匹配规则是随着时间累积、不断加入新的复杂曲折而形成的，它们复杂、甚至可以说有点粗糙 (kludgy)。编程第一定律适用于函数定义和调用，也适用于其他一切地方：保持简单，除非必须复杂。如果你必须为参数排序规则绞尽脑汁才能看懂一段代码，那大概是时候重新评估一下优先级

了。

深度理解

- 核心概念：Lutz 的"降压阀"——规则复杂是因为历史上一步一步加的（老语法 → 3.0 的仅关键字 → 3.5 的多星解包 → 3.8 的仅位置），每个新特性都叠加在老规则上。
- 为什么这样设计：向后兼容优先——Python 不能为"干净"牺牲已发布代码。这是语言演进中"永远不破坏现有代码"（don't break users）哲学的代价。
- 实操建议：日常代码用简单签名（位置 + 默认值 + 少量关键字）；只有库作者才需要掌握全部段位；读代码读不懂时，先怀疑代码本身写得过度复杂，而不是怀疑自己。
- 初学者误区：把"能混排各种模式"当成炫技——真正的功力在于写简单的签名。PEP 8 与 Zen of Python 的 "Simple is better than complex"（简单优于复杂）在此完全适用。

18.15 示例：min 唤醒测试 (Example: The min in Wakeup Call)

英文原文

OK—it's time for something more realistic. To make the concepts here more concrete, the rest of this chapter works through a set of examples that demonstrate practical applications of argument-matching tools.

First up is an exercise borrowed from live classes and used to rouse learners like you starting to succumb to the knottiness of argument rules. Here's the problem statement: suppose you're asked to code a function that is able to compute the minimum value from an arbitrary set of arguments, which may be arbitrary sorts of objects. That is, the function should accept zero or more arguments—as many as you wish to pass. Moreover, the function should work for all kinds of Python object types: numbers, strings, lists, lists of lists, files, and even None. To keep this fair, you don't need to support dictionaries or mixed nonnumeric types, because neither supports direct comparisons, per Chapters 8 and 9. The first requirement provides a natural example of how the feature can be put to good use—we can handle arbitrary arguments by collecting them in a tuple, and stepping over each with a simple for loop. The second part of the problem definition is easy in Python: because nearly every object type supports comparisons, we don't have to specialize the function per type (an application of polymorphism); we can simply compare objects blindly and let Python worry about what sort of comparison to perform according to the objects being compared.

中文翻译

好了——是时候上点更真实的东西了。为了让这里的概念更具体，本章剩下的内容会通过一组示例来演示参数匹配工

具的实际应用。第一个是一个借自线下课堂的练习，用来唤醒像你这样开始被参数规则的错综复杂击倒的学习者。问题陈述如下：假设要求你编写一个函数，能够从任意一组参数中计算出最小值（minimum value），而这些参数可以是任意种类的对象。也就是说，函数应该接受零个或多个参数——你愿意传多少都行。此外，函数应该适用于各种 Python 对象类型：数字、字符串、列表、列表的列表、文件，甚至 None。为了公平起见，你不需要支持字典或混合的非数值类型，因为按第 8 章和第 9 章所述，这两者都不支持直接比较。第一个要求正好天然演示了特性的好用处——我们可以把所有参数收集进一个元组，用简单的 for 循环逐个处理。问题定义的第二部分在 Python 中很容易：因为几乎每种对象类型都支持比较，我们不必按类型特化函数（这是多态 polymorphism 的应用）；我们只管盲目地比较对象，让 Python 按照被比较的对象来决定执行哪种比较。

深度理解

·核心概念："任意数量参数"就该用 args；"任意类型对象"就该依赖多态（duck typing）与运算符重载——< 会根据左侧对象的 lt 自动分派。

·底层实现：arg < res 在字节码层面是 COMPAREOP <，Python 会调用左操作数的 lt 方法（或类型对应的富比较槽位）。对字符串比较码点，对列表递归逐元素比较——全是对象自己的事。

·动态类型优势：C/C++ 要为每种类型写重载或模板；Python 只需写一遍，比较语义由运行时按对象分派。这正是"多态"的威力。

·实际问题：零参数调用的边界（异常而非返回值）也演示了"让异常替你做错误处理"的哲学。

·初学者误区：以为"比较"只对数字有效——Python 中任何定义了比较方法（`lt` 等）的对象都可比较；`None` 与数值比较会 `TypeError`，但同类型之间、以及 `None` 与 `None` 之间是合法的。

18.15.1 满分方案 (Full Credit)

英文原文

The following script file, `mins.py` in Example 18-2, shows four ways to code this operation (some of which were suggested by students in a group exercise designed to prevent post-lunch napping): The first function fetches the first argument from its `args` tuple, and traverses the rest by slicing off the first (there's no point in comparing the first object to itself, especially if it might be a large structure).

The second version lets Python pick off the first and rest of the arguments automatically, and so avoids an index and slice; the code is simpler, and may be faster (though it would take many calls to `min`).

The third converts from a tuple to a list with the built-in `list` call and employs the `list.sort` method: the first item has lowest value after an ascending-value sort.

The fourth sorts too, but skips the list conversion (and two lines) by using the `sorted` built-in function.

Python sorting tools are coded in C, so they can be quicker than the other approaches at times, but the linear scans of the first two techniques may often make them faster.

Example 18-2. mins.py

```
python "Find minimum value among all passed arguments of comparable types"
```

```
def min1(args): res = args[0] for arg in args[1:]: if arg < res: res = arg
```

```
return res def min2(first, rest): for arg in rest: if arg < first: first = arg
```

```
return first def min3(args): tmp = list(args) tmp.sort()
```

```
return tmp[0] def min4(args): return sorted(args)[0]
```

```
if __name__ == '__main__': for func in (min1, min2, min3, min4): # Test all 4 functions
```

```
print(func.name + '...') print(func(3, 4.0, 1, 2)) # Mixed numerics
```

```
print(func('bb','aa')) # Strings: code points
```

```
print(func([2, 2], [1, 1], [3, 3])) # Lists: recursive print
```

```
print(func('hack')) # Unpacked characters This script's testing code uses the name attribute we met earlier, along with a for loop to run each function one at a time (remember, functions are objects that work in a tuple too). All four solutions produce the same result when the file is run, so we'll list just the first's output here. Run this file live, or import it as a module and type a few calls to its function interactively to experiment with them on your own (see Chapter 3 for tips on b
```

oth modes):

```
text $ python3 mins.py min1... 1 aa [1, 1] a ...and the  
same for others...
```

Notice that none of these four variants tests for the case where no arguments are passed in. They could, but there's probably no point in doing so here—in all four solutions, Python will automatically raise an exception to signal the error if no arguments are sent. The first variant raises an exception when we try to fetch argument 0, the second when Python detects an argument list mismatch, and the third and fourth when we try to return item 0 post sort.

This is exactly what we want to happen—because these functions support any object (including None), there is no value that we could pass back to designate an error, so we may as well let the exception be raised. There are exceptions to this exceptions rule (e.g., you might test for errors yourself if you'd rather avoid actions that run before reaching the code that triggers an error automatically). But in general—and especially when errors aren't common—it's better to assume that arguments will work in your functions' code, and let Python raise errors for you when they do not.

中文翻译

下面的脚本文件 mins.py (示例 18-2) 展示了编写这个操作的四种方式 (其中一些是学生们在“防午睡”的小组练习中

提出的)：第一个函数从它的 args 元组里取出第一个参数，然后切掉第一个、遍历其余（把第一个对象与自己比较没有意义，尤其当它可能是个大结构时）。第二个版本让 Python 自动取走第一个参数和其余参数，从而避免了索引和切片；代码更简单，也可能更快（尽管要很多次调用才能体现出差异）。第三个版本用内置 list 调用把元组转成列表，并使用列表 sort 方法：升序排序后第一项就是最小值。第四个版本同样排序，但通过使用 sorted 内置函数跳过了列表转换（和两行代码）。Python 的排序工具是用 C 写的，所以它们有时比其他方法更快，但前两种技术的线性扫描往往反而更快。示例 18-2. mins.py

```
python "在所有可比较类型的传入参数中找出最小值"
def min1(args):
    res = args[0]
    # 先取出第一个参数
    for arg in args[1:]:
        # 用切片跳过第一个，遍历其余
        if arg < res:
            # 如果某个参数更小
            res = arg
    # 就更新最小值
    return res
def min2(first, rest):
    # 让 Python 自动拿走第一个
    for arg in rest:
        # 其余参数收集进 rest
        if arg < first:
            # 逐个与 first 比较
            first = arg
    # 更小就更新 first
    return first
def min3(args):
    tmp = list(args)
    # 转成列表 (sort 是列表方法)
    tmp.sort()
    # 升序排序
    return tmp[0]
# 第一项最小
def min4(args):
    return sorted(args)[0]
# sorted 直接返回新列表
for func in (min1, min2, min3, min4):
    # 逐一测试全部 4 个函数
    print(func.name + '...')
    print(func(3, 4.0, 1, 2))
    # 混合数值
    print(func('bb', 'aa'))
    # 字符串：按码点比较
    print(func([2, 2], [1, 1], [3, 3]))
    # 列表：递归比较
    print(func('hack'))
# 解包的字符
这个脚本的测试代码使用我们之前遇见过的 name 属性，并用 for 循环一次一个地运行每个函数（记住，函数也是对象，一样可以放进元组遍历）。四种解决方案运行结果相同，
```

所以我们这里只列出第一个的输出。你可以直接运行这个文件，或把它作为模块导入，然后交互式地敲几次调用，自己动手实验（两种模式的技巧都见第 3 章）：`text $ python 3 mins.py min1... 1 aa [1, 1] a...`其余函数输出相同...注意：这四个变体没有一个测试“没传任何参数”的情形。它们本可以测，但这里大概没必要——四种方案中，如果没传参数，Python 都会自动抛出异常来标示错误。第一个变体在我们试图取参数 0 时抛异常，第二个在 Python 检测到参数列表不匹配时抛异常，第三个和第四个在我们试图在排序后返回第 0 项时抛异常。这正是我们希望发生的事——因为这些函数支持任意对象（包括 None），找不到一个可以用返回值来标示错误的值，所以我们不如就让异常抛出。这条“异常规则”也有例外（例如，如果你想避免到达自动触发错误的代码之前先执行某些动作，可以自己测错误）。但总的来说——尤其是错误不常见时——更好的做法是：在函数代码中假定参数能正常工作，而当它们不能时，让 Python 替你抛错。

深度理解

- 核心概念：同一功能、四种风格——手写比较（min1/min2）vs 排序法（min3/min4）；显式索引进位 vs 依赖 rest 自动分离。
- 底层实现：func('hack') 把字符串'hack' 解包成四个单字符参数'h','a','c','k'（字符串可迭代），再收集成元组进 args；'aa' < 'bb' 走字符串逐码点比较；[2,2] < [1,1] 走列表的字典序比较。四种版本对 None 之外的可比较对象完全通用。

·性能权衡: `sort/sorted` 是 C 实现的 $O(n \log n)$ 排序 (TimSort), 而 `min1/min2` 是纯 Python $O(n)$ 扫描——因此"线性扫描 vs 排序"谁快并不绝对 (书末脚注解解释了 C Python 的排序会利用部分有序性)。性能问题必须用计时来回答, 不能靠直觉。

·"让异常做错误处理": 对空调用返回什么? 没有可返回的哨兵值 (因为包括 `None` 在内任何值都可能被传入), 所以直接让 `args[0]` 抛 `IndexError`——"最好抛出异常而不是静默失败"。

·初学者误区: 给 `min` 类函数传入不可比较的混合类型 (如 `1` 和 `'a'`) 会 `TypeError`——这是 Python 的"鸭子"比较检查, 符合预期; 空参数调用直接在运行时抛异常而不是返回空值。

18.15.2 加分方案 (Bonus Points)

英文原文

You can get bonus points here for changing these functions to compute the arguments' maximum, rather than minimum, value. This one's trivial: the first two versions only require changing `<` to `>`, and the last two simply require that we return `item[-1]` instead of `[0]`. For an extra point, be sure to mod the function name to "max" as well (though this part is strictly optional).

True curve busters might also note that it's possible to generalize a single function to compute either a mi

nimum or a maximum value, by evaluating comparison expression strings with a tool like the `eval` built-in function (described in Python's library manual, and at various appearances here, especially its note in Chapter 10), or by passing in an arbitrary comparison function. Example 18-3 shows how to implement the latter scheme for one of the coding options.

Example 18-3. `minmax.py`

```
python "Find minimum -or- maximum value of arguments"
```

```
def minmax(test, args): res = args[0] for arg in args[1:]: if test(arg, res): res = arg
```

```
return res def lessthan(x, y): return x < y # See also: lambda, eval def grtrthan(x, y): return x > y print(minmax(lessthan, 4, 2, 1, 5, 6, 3)) # Self-test code print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))
```

Running this script prints both minimum and maximum, per its self-test code at the end:

```
text $ python3 minmax.py 1 6
```

Again, functions are just another kind of object, which allows them to be passed into other functions as done here. To make this a `max` (or other) function, for example, we simply pass in the right sort of test function to `minmax`. This may seem like extra work, but the main point of generalizing functions this way—instead of cutting and pasting to change just a single ch

aracter—is that we'll only have one version to change in the future, not two.

中文翻译

这里你还可以拿加分：把这些函数改成计算参数的最大值，而不是最小值。这个太简单了：前两个版本只需把 `<` 改成 `>`，后两个版本只需返回 `[-1]` 项而不是 `[0]`。想要额外的一分，记得同时把函数名改成 `"max"`（不过这部分严格来说是可选的）。真正的学霸还会注意到：可以把单个函数泛化，让它既能算最小值、也能算最大值——方法之一是借助 `eval` 内置函数求值比较表达式字符串（在 Python 库手册中有描述，本书里也多次出现，尤其第 10 章的说明），或者传入一个任意的比较函数。示例 18-3 演示了如何为其中一种编码选项实现后一种方案。示例 18-3. minmax.py

```
python "找出参数中的最小值——或——最大值"
def minmax(test, args):
    res = args[0]
    for arg in args[1:]:
        if test(arg, res):
            # 用传入的 test 函数决定取舍
            res = arg
    return res
def lessthan(x, y):
    return x < y
# 另见：lambda、eval
def grtrthan(x, y):
    return x > y
print(minmax(lessthan, 4, 2, 1, 5, 6, 3))
# 自测代码
print(minmax(grtrthan, 4, 2, 1, 5, 6, 3))
```

按照结尾的自测代码，运行这个脚本会同时打印最小值和最大值：text \$ python3 minmax.py 1 6 再次强调：函数只是另一种对象，这让它们可以像这里这样被传进其他函数。例如，要让这变成 `max`（或其他）函数，我们只需给 `minmax` 传入合适的 `test` 函数。这看起来像额外的工作，但这样泛化函数的主要意义——而不是复制粘贴去改单个字符——在于未来我们只需要修改一个版本，而不是两个。

深度理解

·核心概念：让"比较逻辑"成为可注入的参数——这是策略模式（Strategy Pattern）的最朴素形态，也是函数式编程中"把行为当数据"的思想。

·底层实现：minmax(lessthan, 4, 2, 1, ...) 中 test 形参绑定到 lessthan 函数对象，函数体内 test(arg, res) 就是一次普通调用——函数对象存储在局部槽位里，跟存储整数没有本质区别。

·为什么这样设计：DRY（Don't Repeat Yourself）原则——与其复制 min 改动一个字符变成 max（两份代码，两处将来要维护），不如一份代码 + 一个可变的比较器。eval 方案被作者顺带提及但有安全风险（执行任意字符串），推荐"传函数"方案。

·生活类比：复印店不区分"文字复印"和"照片复印"——你递进一台机器（同一个函数），选个模式（test 函数）就行；而不是"文字用 A 机、照片用 B 机"（复制两份代码）。

·初学者误区：忘记把函数名后加 () 传参——minmax(lessthan, ...) 传的是函数对象本身（正确），minmax(lessthan(...), ...) 传的是比较结果（错误）；以及低估 eval 安全风险。

18.15.3 关键点（The Punch Line）

英文原文

Of course, all this was just a coding exercise. There's really no reason to write min or max functions, because both are built-ins in Python! We met them briefly in Chapter 5 in conjunction with numeric tools, and again in Chapter 14 when exploring iteration contexts. The built-in versions work almost exactly like ours, but they're coded in C for optimal speed and accept either a single iterable or multiple arguments. Still, though it's superfluous in this context, the general coding pattern we used here might be useful in other scenarios.

中文翻译

当然，这一切只是一个编码练习。真的没有理由去写 min 或 max 函数，因为两者在 Python 中都是内置的！我们在第 5 章结合数值工具简短见过它们，又在第 14 章探索迭代上下文时再次遇到。内置版本的工作方式几乎与我们的完全一样，但它们是 C 实现的、速度最优，而且既能接受单个可迭代对象、也能接受多个参数。尽管如此，虽然在这个场景里它是多余的，我们这里使用的通用编码模式在其他场景中可能很有用。

深度理解

·核心概念：练习的价值在于模式而非结果——“任意参数收集 + 多态比较 + 策略注入”这套组合拳，是写任何泛型工具的模板。

·现状对照：内置 min/max 还能接受关键字参数 (key=、default= 等)，这正是本章前面讲过的"默认值 + 仅关键字参数"的应用实例——读一遍 help(min)，你会发现本章所有语法都在里面。

·为什么这样设计：min/max 用 C 实现 + 同一套参数匹配框架 (args 收集、可选关键字)，证明"内置函数也是按同一套规则工作的"——你学到的语法能直接读懂标准库。

·初学者误区：以为"自己写了 min 就可以乱用"——请直接用内置，内置更快、更全（支持 key/default）、且经过无数测试。

18.16 示例：通用集合函数 (Example: Generalized Set Functions)

英文原文

Our next example also demos special argument-matching modes at work. At the end of Chapter 16, we wrote a function that returned the intersection of two sequences (really, it picked out items that appeared in both). Example 18-4 codes an augmented version that intersects an arbitrary number of sequences (one or more) by using the argument-matching form args to collect all the passed-in arguments.

Because the arguments come in as a tuple, we can process them in a simple for loop. Just for fun, we'll co

de a union function that also accepts an arbitrary number of arguments to collect items that appear in any of the operands.

Example 18-4. inter2.py

python """Implement intersection and union for one or more arguments. Inputs may be any sort of iterable that supports multiple in tests, and results are always lists. This intersect avoids duplicates in results by in test, but may be slow: improve me.

```
"""  
  
def intersect(args): res = [] for x in args[0]: # Scan first  
sequence if x in res: continue # Skip duplicates in [0  
] for other in args[1:]: # For all other args if x not in o  
ther: break else: # No: break out of loop res.append(x  
) # Yes: add items to end
```

```
return res def union(args): res = [] for seq in args: # F  
or all args for x in seq: # For all in this arg if not x in r  
es: res.append(x) # Add new items to result
```

```
return res
```

Because these tools are potentially worth reusing (and are too big to retype interactively), we'll store the functions in a module file called inter2.py. Again, if you're unsure about how modules and imports work, see the introduction in Chapter 3, or stay tuned for in-depth coverage in Part V. This chapter's module usage is simple, but per Chapter 3, be sure to launch your REPL in the same folder as the file, so imp

orts can find it.

Like Chapter 16's original `intersect`, both of the functions in this module work on any kind of sequence. Here they are live at the REPL, processing strings, mixed types, and more than two sequences:

```
text $ python3 >> from inter2 import intersect, union
n >> s1, s2, s3 = 'HACKK', 'CODE', 'CASH' >> intersect(s1, s2), union(s1, s3) # Two operands (['C'], ['H', 'A', 'C', 'K', 'S'])
>> intersect([1, 2, 3, 4], (1, 4)) # Mixed types [1, 4]
>> intersect(s1, s2, s3) # Three operands ['C']
> union(s1, s2, s3) ['H', 'A', 'C', 'K', 'O', 'D', 'E', 'S']
```

中文翻译

下一个示例同样演示特殊参数匹配模式的实战。在第 16 章末尾，我们写过一个返回两个序列交集（intersection）的函数（实际上，它挑出同时出现在两者中的元素）。示例 18-4 编写的增强版使用参数匹配形式 `args` 收集所有传入参数，从而能够求任意数量（一个或多个）序列的交集。因为参数以元组形式进来，我们可以用简单的 `for` 循环处理它们。顺便为了好玩，我们再写一个 `union` 函数，它也接受任意数量的参数，用来收集出现在任何一个操作数中的元素。示例 18-4. `inter2.py`

```
python """为一个或多个参数实现交集与并集。输入可以是任何支持多重 in 测试的可迭代对象，结果总是列表。这个 intersect 通过 in 测试避免结果中的重复项，但可能较慢：请改进我。"""
def intersect(args):
    res = []
    for x in args[0]: # 扫描第一个序列
        if x in res:
```

```

continue # 跳过 [0] 中的重复项
for other in args[1:]: # 遍历所有其他参数
    if x not in other: break # 若不在某一序列中则跳出
else: # 没有跳出（在每一个里都有）
    res.append(x) # 是：把元素追加到末尾
return res

def union(args):
    res = []
    for seq in args: # 遍历所有参数
        for x in seq: # 遍历这个参数里的全部元素
            if not x in res:
                res.append(x) # 把新元素加入结果
    return res

```

因为这些工具值得复用（而且太大，不适合在交互式环境里重敲），我们会把函数存进名为 `inter2.py` 的模块文件。再说一次，如果你对模块和导入的工作方式没把握，请看第 3 章的开头介绍，或等着第五部分的深入讲解。本章的模块用法很简单，但按第 3 章所述，务必在与文件相同的文件夹里启动你的 REPL，这样 `import` 才能找到它。和第 16 章的原始 `intersect` 一样，这个模块中的两个函数都适用于任何种类的序列。下面是它们在 REPL 中活生生的表现，处理字符串、混合类型以及两个以上的序列：

```

text $ python3 >>> from inter2 import intersect, union
>>> s1, s2, s3 = 'HACKK','CODE','CASH'
>>> intersect(s1, s2), union(s1, s3) # 两个操作数(['C'], ['H','A','C','K','S'])
>>> intersect([1, 2, 3, 4], (1, 4)) # 混合类型
[1, 4]
>>> intersect(s1, s2, s3) # 三个操作数['C']
>>> union(s1, s2, s3) ['H','A','C','K','O','D','E','S']

```

深度理解

·核心概念：args 把“任意数量的序列”收成一个元组，让算法用最朴素的循环就能处理 N 个操作数；for ... else 子句用来判断“是否在所有序列中都出现”。

·底层实现：x in res 走序列的 contains（列表是线性扫描）；字符串的 in 是子串匹配（所以对字符序列结果依然

正确)；字典的 `in` 是键查找。算法时间复杂度 $O(n \cdot m)$ ，作者在 `docstring` 里大方承认 "may be slow: improve me"。

·一个设计细节：`intersect` 用 `if x in res: continue` 去重——避免了重复元素进入结果；这正是“集合操作”语义（集合无重复）与“序列操作”的区别。

·为什么这样设计：模块文件 = 可复用工具库的雏形；REPL 快速实验 + 模块持久化 = Python 的典型开发 workflow。

·初学者误区：忽略 `for...else`——这里的 `else` 在循环没有被 `break` 中断时执行；把 `intersect(s1, s2)` 与 `{'C'}` 混淆——结果是列表 `['C']`，不是集合。

18.16.1 测试代码 (Testing the Code)

英文原文

To test more thoroughly, the following continues our REPL session to code a function that applies the two tools to arguments in different orders using a simple shuffling technique that we saw in Chapter 13—it slices to move the first to the end on each loop, uses a `tuple` to unpack arguments, and sorts so results are comparable. Notice that arguments for the function are sent as a sequence (not discrete items), and the trace configuration option is keyword-only here:

```
python >> def tester(func, items, , trace=True):
... for i in range(len(items)): ... items = items[1:] + items[:1] # Move front item to back ... if trace: print(items)
```

```
ms) ... print(sorted(func(items))) # Test with reordered items
>> tester(intersect, (s1, s2, s3)) # Use strings from prior listing
('CODE','CASH','HACKK') ['C'] ('CASH','HACKK','CODE') ['C']
('HACKK','CODE','CASH') ['C']
>> tester(union, (s1, s2, s3), trace=False) ['A','C','D','E','H','K','O','S']
['A','C','D','E','H','K','O','S'] ['A','C','D','E','H','K','O','S']
>> tester(intersect, (s1, s2, s3), trace=False) ['C'] ['C'] ['C']
```

Two context notes here: first, because duplicates won't appear in these intersection and union functions, they qualify as set operations mathematically, but may not be optimal in term of speed. Still, there's not much point in improving this demo's code—intersection and union, like min and max, are built-in operations today: the set object we explored in Chapter 5 does intersection and union with `&` and `|`, and has methods that take multiple operands too. Hence, optimizing this code is left as suggested exercise, but see `inter3.py` in the examples package for pointers.

Second, the argument scrambling in the tester here doesn't generate all possible argument orders (that would require a full permutation, and 6 orderings for 3 arguments), but suffices to check if argument order impacts results. As a preview, though, the tester would be simpler and more flexible if it delegated scrambling to a reusable function. Watch for this revision in Chapter 20, after we've explored how to code user-defined generators. We'll also recode set tools one last

t time in Chapter 32 and a solution to a Part VI exercise, as classes that add them to the list object as methods.

中文翻译

为了更彻底地测试，下面接着 REPL 会话，编写一个使用第 13 章学过的简单洗牌 (shuffling) 技巧、把两个工具以不同顺序应用于参数的函数——它每次循环用切片把第一个挪到末尾，用一个解包参数，并对结果排序以便比较。注意：函数的参数是以序列（而非离散项）发送的，而且这里的 trace 配置选项是仅关键字参数：python >>> def tester(func, items, , trace=True): ... for i in range(len(items)): ... items = items[1:] + items[:1] # 把最前面的项挪到末尾 ... if trace: print(items) ... print(sorted(func(items))) # 用重排后的项测试>>> tester(intersect, (s1, s2, s3)) # 使用前一个清单里的字符串('CODE','CASH','HACKK') ['C'] ('CASH','HACKK','CODE') ['C'] ('HACKK','CODE','CASH') ['C'] >>> tester(union, (s1, s2, s3), trace=False) ['A','C','D','E','H','K','O','S'] ['A','C','D','E','H','K','O','S'] ['A','C','D','E','H','K','O','S'] >>> tester(intersect, (s1, s2, s3), trace=False) ['C'] ['C'] ['C'] 这里有两个上下文说明：第一，因为重复项不会出现在这些交集与并集函数的结果中，所以它们在数学上称得上集合操作，但速度上可能不是最优的。不过，改进这个演示代码没有多大意义——交集和并集，像 min 和 max 一样，如今已是内置操作：我们在第 5 章探索的 set 对象用 & 和 | 做交集和并集，而且还有接受多个操作数的方法。因此，优化这里的代码留作建议练习，但可以看示例包中的 inter3.py 获取提示。第二，测试器里的

参数打乱并没有生成所有可能的参数顺序（那需要完整排列，3 个参数有 6 种顺序），但足以检查参数顺序是否影响结果。不过提前预告一下：如果测试器把打乱工作委托给一个可复用的函数，它会更简单、更灵活。第 20 章在我们探索如何编写用户自定义生成器之后，会看到这个修订版。我们还会在第 32 章以及第六部分某练习的解答中，最后一次把集合工具重写为类，把它们作为方法添加到列表对象上。

深度理解

- 核心概念：`tester(func, items, , trace=True)` 是仅关键字参数在测试/工具函数中的教科书用法——`trace` 是"配置"，与数据 `items` 隔离；`func(items)` 是"解包转发"。
- 底层实现：`items = items[1:] + items[:1]` 每次把元组切成两段再拼接，形成轮换（rotation）序列——`('HACKK', 'CODE', 'CASH') → ('CODE', 'CASH', 'HACKK') → ...`；`sorted(func(items))` 把结果列表排序以保证可比对。
- 为什么这样设计：测试要验证"参数顺序是否影响结果"——使用轮换而非全部排列，是 06 与 01 的成本权衡。集合语义的测试目标明确：顺序不应改变交集/并集。
- 现代修正：Lutz 坦言如今应直接用 `set (set1 & set2 & set3` 或 `set1.intersection(...)`、`.union(...)`；本章例子的历史意义是展示 `args` 与序列处理，而不是替代内置。
- 科学精神：测试要排序结果再比较——因为"顺序无关性"本身也是被测属性之一；不排序直接比列表相等，反而会误报。
- 初学者误区：把 `tester` 的元组轮换误以为是可变操作——

一元组不可变，`items[1:] + items[:1]` 每次都生成新元组并重新绑定 `items`；漏掉会把整个元组传给 `func`。

18.17 示例：自己动手写 `print` (Example: Rolling Your Own Print)

英文原文

To close out the chapter, let's look at one last example of argument matching at work: this section uses both and arbitrary-arguments collectors up front, and keyword-only arguments later, to emulate most of what Python's `print` function does. Like the preceding examples, there's no urgent reason to code tools that Python provides. Also like the others, though, this is both instructive and may be a basis for custom variants of built-in tools.

For both purposes, the module file in Example 18-5 does roughly the same job as `print` in a small amount of reusable and modifiable code, by building and routing the print string per configuration arguments.

Example 18-5. `print3.py`

```
python r"""Emulate most of the Python 3.X print function as customizable code. Call signature: print3(args, sep='', end='\n', file=sys.stdout).
```

```
"""
```

```
import sys
def print3(args, kargs):
    sep = kargs.get('sep', '') # Keyword arg defaults
    end = kargs.get('end', '\n')
    file = kargs.get('file', sys.stdout)
    output = '' # Build a string
    first = True
    for arg in args:
        output += (arg if first else sep) + str(arg)
        first = False
    file.write(output + end)
```

Notice that this module's docstring uses a raw string to retain its backslash for help (per Chapter 15). Also note that this module's function need not be called "print3" because "print" is a built-in but not a reserved word, but using a different name avoids inadvertently hiding the built-in. To test it, import this into another file or the interactive prompt, and use it like the print built-in.

Example 18-6 codes a test script that imports our printer as a demo.

Example 18-6. test-print3.py

```
python from print3 import print3
print3(1, 2, 3) # Defaults
print3(1, 2, 3, sep='') # Suppress separator
print3(1, 2, 3, sep='...') # Custom separator
print3(1, [2], (3,), sep='...') # Various object types
print3(4, 5, 6, sep='', end='') # Suppress newline
print3(7, 8, 9) # Finish line
print3() # Blank line
```

```
import sys
print3(1, 2, 3, sep='?', end='.\n', file=sys.stderr) # Redirect to stream
print3('LP6E was here', file=open('log.txt', 'w')) # Redirect to a file
print3(open('l
```

```
og.txt').read())
```

When this is run, our `print3` produces the same results as the `print` built-in. Fine points here: `stderr` goes to your console by default, and it's OK to use a dash in this script's name because it's run, not imported (again, we'll be focusing on such module details in this book's next part):

```
text $ python3 test-print3.py 1 2 3 123 1...2...3 1...[2]..
.(3,) 4567 8 9 1?2?3. LP6E was here
```

As usual, the generality of its toolset allows us to prototype or develop concepts in the Python language itself. In this case, argument-matching tools are as flexible in Python code as they are in Python's internal implementation.

中文翻译

为了给本章收尾，让我们看最后一个参数匹配实战示例：本节先用和任意参数收集器，后面再用仅关键字参数，来模拟 Python `print` 函数的大部分功能。与前面的示例一样，没有紧迫的理由去编写 Python 已经提供的工具。但与其他示例一样，它既有教学意义，也可能成为内置工具定制变体的基础。出于这两个目的，示例 18-5 中的模块文件用少量可复用、可修改的代码做了与 `print` 大致相同的工作——按配置参数构建并路由打印字符串。示例 18-5. `print3.py` python r"""以可定制的代码模拟 Python 3.X `print` 函数的大部分功能。调用签名：`print3(args, sep=',', end='\n', file=sys.stdout)`。""" import sys def print3(args, kargs): sep

```
= kargs.get('sep','') # 关键字参数的默认值
end = kargs.get('end','\n')
file = kargs.get('file', sys.stdout)
output = '' # 构建+打印一个字符串
first = True
for arg in args:
    output += (' if first else sep) + str(arg)
    first = False
file.write(output + end)

```

注意：这个模块的文档字符串（doc string）使用了原始字符串（raw string），以便在 help 时保留其反斜杠（见第 15 章）。还要注意：这个模块的函数不必叫 "print3"，因为 "print" 是内置函数而不是保留字，但换个名字可以避免无意中遮蔽（hiding）内置函数。要测试它，把它导入另一个文件或交互式提示符，然后像使用 print 内置函数一样使用它。示例 18-6 编写了一个测试脚本，把我们的打印机导入作为演示。

示例 18-6. test-print3.py

```
python from print3 import print3
print3(1, 2, 3)
# 默认值
print3(1, 2, 3, sep='') # 抑制分隔符
print3(1, 2, 3, sep='...') # 自定义分隔符
print3(1, [2], (3,), sep='...') # 各种对象类型
print3(4, 5, 6, sep='', end='') # 抑制换行
print3(7, 8, 9) # 补完这一行
print3() # 空行
import sys
print3(1, 2, 3, sep='?', end='.\n', file=sys.stderr) # 重定向到流
print3('LP6E was here', file=open('log.txt','w')) # 重定向到文件
print3(open('log.txt').read())

```

运行时，我们的 print3 产生与 print 内置函数相同的结果。这里有几个细节：stderr 默认输出到你的控制台；脚本名里使用连字符（dash）没问题，因为它是被运行而不是被导入的（再说一次，本书下一部分会重点讲这类模块细节）：

```
text $ python3 test-print3.py 1 2 3 123 1...2...3 1...[2]...(3,) 456 7 8 9 1?2?3. LP6E was here
```

像往常一样，其工具集的通用性让我们可以在 Python 语言本身里做原型或开发概念。在这个案例中，参数匹配工具在 Python 代码中的灵活程度

，与它们在 Python 内部实现中的灵活程度一样高。

深度理解

·核心概念：用 args, kargs 重造 print——这是"拆解标准库"的最佳教学：你会意识到 print 的签名其实就是 `print(objects, sep=',', end='\n', file=sys.stdout, flush=False)`。

·底层实现：`kargs.get('sep', '')` 从 kargs 字典取配置，缺省用 Python 级默认值——这正是"kwargs 手动管理默认值"方案的典型代码；而真正的 print 内置用 C 的 PyArg Parser 直接解析关键字。

·原始字符串：docstring 写成 `r"""...\n..."""`，`\n` 不会被转义成换行，`help(print3)` 时用户能看到字面 `\n`——文档里要展示反斜杠时的标准技巧。

·为什么不用 "print" 命名：print 是内置名不是关键字；`def print(...)` 会在当前模块遮蔽内置，之后本模块内想用真 print 就麻烦了——换名或 `import builtins` 是正解。

·实际问题：文件路径含连字符 (`test-print3.py`) 无法被 `import` (`import` 语句要求合法标识符)，但可以作为顶层脚本运行——这解释了为什么文件名规则要区分"运行的脚本"与"导入的模块"。

·初学者误区：`file=sys.stderr` 传的是流对象（不是文件名）；`print3(open('log.txt').read())` 是"先读文件再打印"，与 `file=` 写入方向相反——一个进一个出。

18.17.1 使用仅关键字参数 (Using Keyword-Only Arguments)

英文原文

Our print emulator works, but has a minor flaw baked in: it assumes that all positional arguments are to be printed, and all keywords are for options only. Any extra keyword arguments are silently ignored, and neither printed nor reported. A call like the following, for instance, will ignore the extra—and likely erroneous—`sap` argument:

```
text $ python3 >> from print3 import print3 >> print3(3.12,'py', sap='@') 3.12 py
```

It may not make sense to print superfluous keywords, but we can detect them manually by using `dict.pop()` to delete fetched keywords, and checking if the dictionary is not empty when we're done. The version in Example 18-7 does—it's equivalent to the original, but triggers a built-in exception with a `raise` statement when unexpected keyword arguments are sent by a call (this is partly preview: we'll study exceptions and `raise` in depth in Part VII).

Example 18-7. `print3pops.py`

```
python "Use keyword-collector arguments with deletion and defaults"
```

```
import sys def print3(args, kargs): sep = kargs.pop('s
ep',') end = kargs.pop('end','\n') file = kargs.pop('file
', sys.stdout) if kargs: raise TypeError(f'extra keywords
: {kargs}') output = "first = True for arg in args: output
 += (" if first else sep) + str(arg)

first = False file.write(output + end)
```

Notice that this file's name uses an underscore instead of a dash: because it's to be imported, its name must follow the rules for variables of Chapter 11. This version works as before, but it now catches extraneous keyword arguments:

```
python >> from print3pops import print3 >> print3(
3.12,'py', sep='@') 3.12@py >> print3(3.12,'py', sap='
@') TypeError: extra keywords: {'sap': '@'}
```

It's OK to reimport the same `print3` name from a different file here: this simply replaces the prior version just like reassigning any other variable (more on this later in this book). That being coded, this example could also use keyword-only arguments to automatically validate configuration arguments, as the final variant in Example 18-8 illustrates.

Example 18-8. `print3kwonly.py`

```
python "Use keyword-only arguments to emulate print"
```

```
import sys def print3(args, sep="", end="\n", file=sys.s
tdout): output = "first = True for arg in args: output +
```



```
= (' if first else sep) + str(arg)
```

```
first = False file.write(output + end)
```

This version works the same way for valid calls, but catches invalid keywords with keyword-only arguments instead of manual code, and is a prime example of how keyword-only arguments can address coding needs:

```
python >> from print3kwonly import print3 >> print3(3.12,'py', sep='@') 3.12@py >> print3(3.12,'py', sap='@')
TypeError: print3() got an unexpected keyword argument'sap'
```

Given that this version of the function also requires four fewer lines of code than its predecessor, keyword-only arguments can simplify a specific category of functions that accept both arguments and options. A similar case can be made for positional-only arguments versus manual code, but it's more obscure, and this chapter has run out of space. For another example of keyword-only arguments at work, stay tuned for the iteration-timing case study in Chapter 21.

And for more inspiration, also see the sidebar "Why You Will Care: Customizing open". Much as we did there, our print emulator could be assigned to `builtins.print` to replace the built-in with our custom version everywhere in a program. There's no reason to do that for a version that's the same, of course, but this technique can be used to install a replacement printer that

mods or extends the built-in (e.g., with logging).

中文翻译

我们的 print 模拟器能工作，但内置了一个小瑕疵：它假定所有位置参数都是要打印的，所有关键字都只是选项。任何多余的关键字参数都会被静默忽略，既不打印也不报告。例如，像下面这样的调用会忽略多余——而且很可能是错误的——sap 参数：

```
text $ python3 >>> from print3 import print3 >>> print3(3.12,'py', sap='@') 3.12 py
```

打印多余的 (superfluous) 关键字可能没意义，但我们可以手动检测它们：用 dict.pop() 删除已取走的关键字，然后检查字典在处理完后是否非空。示例 18-7 的版本就是这么做的——它与原版等价，但当调用传入意外关键字参数时，会用 raise 语句触发内置异常（这部分是预告：我们将在第七部分深入研究异常与 raise）。示例 18-7. print3pops.py

```
python "使用带删除与默认值的关键字收集器参数" import sys def print3(args, kargs): sep = kargs.pop('sep','') # 取走并删除 sep, 缺省空格 end = kargs.pop('end','\n') file = kargs.pop('file', sys.stdout) if kargs: raise TypeError(f'extra keywords: {kargs}') # 还有剩的? 报错 output = "first = True for arg in args: output += (' if first else sep) + str(arg) first = False file.write(output + end)
```

注意：这个文件名用的是下划线而不是连字符——因为它要被导入，其名字必须遵循第 11 章的变量命名规则。这个版本和之前一样工作，但现在它会捕获多余的关键字参数：

```
python >>> from print3pops import print3 >>> print3(3.12,'py', sep='@') 3.12@py >>> print3(3.12,'py', sap='@') TypeError: extra keywords: {'sap': '@'} 这里从
```

不同文件重新导入同一个 `print3` 名字是没问题的：它只是替换掉之前的版本，就像重新赋值任何其他变量一样（本书后面会详述）。既然写到了这里，这个例子其实还可以用仅关键字参数来自动校验配置参数，正如示例 18-8 的最终变体所示。示例 18-8. `print3kwonly.py`

```
python "使用仅关键字参数模拟 print"
import sys
def print3(args, sep=" ", end="\n", file=sys.stdout):
    output = "first = True for arg in args: output += (' if first else sep) + str(arg) first = False file.write(output + end)
    这个版本对合法调用的工作方式相同，但它用仅关键字参数（而不是手写代码）来捕获非法关键字，是“仅关键字参数如何解决编码需求”的最佳范例：
```

```
python >>> from print3kwonly import print3
>>> print3(3.12,'py', sep='@') 3.12@py
>>> print3(3.12,'py', sap='@')
TypeError: print3() got an unexpected keyword argument 'sap'
```

考虑到这个版本的函数还比它的前身少了四行代码，可以说仅关键字参数能简化一类特定函数——那些既要接受参数又要接受选项的函数。类似的论证也可以用在仅位置参数对比手写代码上，但它更冷门，而本章的篇幅已经不够了。仅关键字参数实战的另一个例子，敬请期待第 21 章的迭代计时案例研究。更多灵感，还可以看看侧边栏“为什么你会在意：自定义 `open`”。和我们在那里的做法差不多，我们的 `print` 模拟器可以赋给 `builtins.print`，在程序中的任何地方用自定义版本替换内置函数。当然，对一个相同的版本没有理由这么做，但这个技巧可以用来安装一个修改或扩展内置功能的替换打印机（比如加上日志）。

深度理解

·核心概念：三个版本对比，一条进化主线——kargs 手动管理（容忍错误键）→ dict.pop() 手动校验（多了 4 行代码）→ 仅关键字参数自动校验（少 4 行代码，错误键自动 TypeError）。结论：能用语法解决的问题不要用代码解决。

·底层实现：kargs.pop('sep','') 用默认参数执行 dict.pop（键存在则删除并返回，否则返回默认值）；if kargs: 检查剩余键——pop 后字典非空说明有未知配置。仅关键字版本则把校验交给解释器的参数匹配器，运行期直接报 TypeError: got an unexpected keyword argument'sap'。

·为什么这样设计：静默吞错（silent failure）是最坏的行为模式——sap='@' 被忽略意味着拼写错误永远不会被察觉；raise TypeError 是“快速失败”（fail fast）哲学的体现。

·替换内置：builtins.print = print3（或 main 作用域里 print = print3）即可全局替换——这是 monkey patching 的最简单形态，是“为什么你会在意”侧边栏（18.20 节）预告的主题。

·初学者误区：kargs.get() 与 kargs.pop() 的区别——get 不删除键，如果后面检查 if kargs 就永远不报错；把 file= 误写成文件名而非流对象；用 print3 的旧版本重命名遮蔽内置导致“print 不工作了”的困惑。

18.18 本章小结 (Chapter Summary)

英文原文

In this chapter, we studied the second of two key concepts related to functions: the arguments used to send objects to a function. As we saw, arguments are passed to a function by assignment, which means by object reference (which really means by pointer), and are open to the usual side effects for shared mutable objects, desired or not. We also studied some advanced extensions that generalize argument matching, including default and keyword arguments, tools for collecting and unpacking arbitrarily many arguments, and keyword- and positional-only arguments. Finally, we explored a few larger examples that employed argument tools, and previewed module topics of this book's next part. The next chapter continues our look at functions with a grab bag of more advanced function-related ideas: function annotations, recursion, and more on lambda and functional tools such as map and filter. Many of these concepts stem from the fact that functions are normal objects in Python, and so support flexible processing tools and modes. Before diving into those topics, however, take this chapter's quiz to review the argument ideas we've studied here.

中文翻译

在本章中，我们研究了与函数相关的两个关键概念中的第二个：用于把对象发送给函数的参数（arguments）。如我们所见，参数按赋值传给函数——也就是按对象引用（实际上意味着按指针）传递，并且会遭遇共享可变对象常见的副作用，无论你愿不愿意。我们还研究了一些泛化参数匹配的高级扩展，包括默认参数和关键字参数、收集与解包任意数量参数的工具，以及仅关键字和仅位置参数。最后，我们探索了几个使用参数工具的较大示例，并预告了本书下一部分的模块主题。下一章将以“高级函数相关想法的杂货袋”继续我们对函数的探索：函数注解（function annotations）、递归（recursion），以及更多关于 lambda 和 map、filter 等函数式工具的内容。这些概念中很多源于一个事实：函数在 Python 中就是普通对象，因此支持灵活的处理工具与模式。不过，在投入这些主题之前，先做做本章的测验，复习一下我们在这里学到的参数思想。

深度理解

·本章知识地图：① 参数传递的本质（按赋值/传引用三定律）→ ② 匹配模式的六件工具（位置、关键字、默认值、/、仅关键字、仅位置）→ ③ 排序规则（定义侧四段式、调用侧三段式）→ ④ 三个综合案例（min、集合函数、print 模拟器）。

·与后续章节的连接：第 19 章的注解（annotation）不影响排序规则；map/filter 大量依赖“函数即对象”；第 39 章的装饰器是 args, kwargs 转发模式的大本营。

一句话总结：参数的背后是赋值，赋值背后是引用，引用背后是指针——三层递进，全部理解即可通关。

18.19 测验：检验你的知识 (Test Your Knowledge: Quiz)

英文原文

This quiz asks you to trace through examples of function definitions and calls to predict their outputs. Try to work out the answers on your own before resorting to cut and paste in a REPL: 1. What is the output of the following code, and why?

```
python >> def func(a, b=4, c=5):
```

```
... print(a, b, c) >> func(1, 2)
```

2. What is the output of this code, and why?

```
python >> def func(a, b, c=5):
```

```
... print(a, b, c) >> func(1, c=3, b=2)
```

3. How about this code: what is its output, and why?

```
python >> def func(a, pargs):
```

```
... print(a, pargs) >> func(1, 2, 3)
```

4. What does this code print, and why?

```
python >> def func(a, kargs):
```

```
... print(a, kargs) >> func(a=1, c=3, b=2)
```

5. What gets printed by this, and why?

```
python >> def func(a, b, c=3, d=4): print(a, b, c, d) >  
> func(1, (5, 6))
```

6. One last time: what is the output of this code, and why?

```
python >> def func(a, b, c):  
... a = 2; b[0] = 'x'; c['a'] = 'y'>> L=1; M=[1]; N={'a': 0}  
>> func(L, M, N) >> L, M, N
```

中文翻译

这个测验要求你追踪函数定义与调用的示例，预测它们的输出。请先尝试自己独立得出答案，再去 REPL 里复制粘贴：

1. 下面代码的输出是什么，为什么？
python >>> def func(a, b=4, c=5): ... print(a, b, c) >>> func(1, 2)
2. 这段代码的输出是什么，为什么？
python >>> def func(a, b, c=5): ... print(a, b, c) >>> func(1, c=3, b=2)
3. 这段代码呢：输出是什么，为什么？
python >>> def func(a, pargs): ... print(a, pargs) >>> func(1, 2, 3)
4. 这段代码打印什么，为什么？
python >>> def func(a, kargs): ... print(a, kargs) >>> func(a=1, c=3, b=2)
5. 这段代码打印什么，为什么？
python >>> def func(a, b, c=3, d=4): print(a, b, c, d) >>> func(1, (5, 6))
6. 最后一次：这段代码的输出是什么，为什么？
python >>> def func(a, b, c): ... a = 2; b[0] = 'x'; c['a'] = 'y'>>> L=1; M=[1]; N={'a': 0} >>> func(L, M, N) >>> L, M, N

测验提示 (深度理解)

·第 1-2 题考"位置匹配 + 默认值"; 第 3 题考 pargs 收集; 第 4 题考 kargs 收集与键序; 第 5 题考调用侧解包与默认值; 第 6 题考"重绑定名字 vs 原地修改对象"三定律。全部做对, 说明本章已通关。

·第 5 题有一个隐蔽点: (5, 6) 被解包成两个位置参数, 按位置进入 b 和 c——覆盖了 c 的默认值 3, 而 d 没有被赋值, 仍取默认值 4。

·第 6 题的坑: L=1 是整数 (不可变), a = 2 只重绑定局部名字; M 是列表、N 是字典 (可变), b[0] = 'x'、c['a'] = 'y' 是原地修改——调用方看到的对象内容变了。

18.20 测验答案 (Test Your Knowledge: Answers)

英文原文

1. The output here is 1 2 5, because 1 and 2 are passed to a and b by position, and c is omitted in the call and hence defaults to 5. 2. The output this time is 1 2 3: 1 is passed to a by position, and b and c are passed 2 and 3 by name (the left-to-right order doesn't matter when keyword arguments are used like this). 3. This code prints 1 (2, 3), because 1 is passed to a and the pargs collects the remaining positional arguments into a new tuple object. We can step through the ex

tra positional arguments tuple with any iteration tool (e.g., `for arg in pargs: ...`). 4. This time the code prints `1 {'c': 3, 'b': 2}`, because 1 is passed to a by name and the kargs collects the remaining keyword arguments into a dictionary. We could step through the extra keyword arguments dictionary by key with any iteration tool (e.g., `for key in kargs: ...`). Note that the order of the dictionary's keys reflects the order in which keyword arguments are passed, in recent Pythons. 5. The output here is `1 5 6 4`: the 1 matches a by position, 5 and 6 match b and c by positional unpacking (6 overrides c's default), and d defaults to 4 because it was not passed a value. 6. This displays `(1, ['x'], {'a': 'y'})`—the first assignment in the function doesn't impact the caller, but the second two do because they change passed-in mutable objects in place.

中文翻译

1. 这里的输出是 `1 2 5`: 因为 1 和 2 按位置传给 a 和 b, 而 c 在调用中被省略, 所以默认值为 5。2. 这次输出是 `1 2 3`: 1 按位置传给 a, b 和 c 分别按名字传入 2 和 3 (这样使用关键字参数时, 从左到右的顺序无关紧要)。3. 这段代码打印 `1 (2, 3)`: 因为 1 传给 a, pargs 把剩余位置参数收集进一个新的元组对象。我们可以用任何迭代工具遍历这个多余位置参数元组 (例如 `for arg in pargs: ...`)。4. 这次代码打印 `1 {'c': 3, 'b': 2}`: 因为 1 按名字传给 a, kargs 把剩余关键字参数收集进一个字典。我们可以用任何迭代工具按键遍历这个多余关键字参数字典 (例如 `for key in kar`

gs: ...)。注意：在近期的 Python 版本中，字典键的顺序反映了关键字参数被传入的顺序。5. 这里的输出是 1 5 6 4 : 1 按位置匹配 a, 5 和 6 通过位置解包匹配 b 和 c (6 覆盖了 c 的默认值), 而 d 因为没有传值、默认取 4。6. 这里显示 (1, ['x'], {'a': 'y'})——函数中的第一个赋值不影响调用方, 但后两个赋值会影响, 因为它们原地修改了传入的可变对象。

18.21 为什么你会在意：关键字参数 (WHY YOU WILL CARE: KEYWORD ARGUMENTS)

英文原文

As you can probably tell, some argument-matching mode combos can be complex. They are also largely optional in your code; in fact, you can get by with just simple positional matching, and it's probably a good idea to do so when you're starting out. However, because many Python tools make good use of them, some general knowledge of these modes is important. For example, keyword arguments play a key role in tkinter, the de facto standard GUI API for Python. We touch on tkinter only briefly at various points in this book, but in terms of its call patterns, keyword arguments set configuration options when GUI components are built. For instance, a call of the form: `python from tkinter import Button widget = Button(text='Press me', command=someFunction)` creates a new button a

nd specifies its text displayed in the GUI and callback function run on a press, using the text and command keyword arguments. Since the number of configuration options for a widget can be large, keyword arguments let you pick and choose which to apply. Without them, you might have to either list all the possible options by position or hope for a judicious positional-argument defaults protocol that would handle every possible option arrangement. Many built-in functions in Python expect us to use keywords for usage-mode options as well, which may or may not have defaults. As we learned in Chapter 8, for instance, the sorted built-in: `python sorted(iterable, key=None, reverse=False)` expects us to pass an iterable object to be sorted, but also allows us to pass in optional keyword arguments to specify both a value-transform function and a reversal flag, which default to `None` and `False`, respectively. Since we normally don't use these options, they may be omitted to apply defaults. As we've also seen, the `dict`, `str.format`, and `print` calls accept keywords as well—other usages we had to introduce in earlier chapters because of their forward dependence on argument-passing modes we've finally studied here (alas, those who change Python already know Python!).

中文翻译

你可能已经感觉到，某些参数匹配模式的组合可以很复杂。它们在你的代码中很大程度上也是可选的；事实上，只用简

单的位置匹配你也能过得去，而且在起步阶段这么做很可能是好主意。然而，因为许多 Python 工具都善用了这些模式，对这些模式有一些一般性了解是很重要的。例如，关键字参数在 tkinter 中扮演关键角色——它是 Python 事实上的标准 GUI API。本书在多个地方只是简短触及 tkinter，但就它的调用模式而言，关键字参数在构建 GUI 组件时用来设置配置选项。例如，这样形式的调用：`python from tkinter import Button widget = Button(text='Press me', command=someFunction)` 创建一个新按钮，并用 `text` 和 `command` 关键字参数指定它在 GUI 中显示的文本、以及按下时运行的回调函数。因为一个组件的配置选项数量可能很大，关键字参数让你可以挑选要应用哪些。没有它们，你可能不得不按位置列出所有可能的选项，或者指望一个能处理所有可能选项排列的、明智的位置参数默认值协议。Python 中的许多内置函数也期望我们用关键字传递"用法模式"选项，这些选项可能有也可能没有默认值。例如，正如我们在第 8 章学到的，`sorted` 内置函数：`python sorted(iterable, key=None, reverse=False)` 期望我们传入一个要排序的可迭代对象，但也允许我们传入可选的关键字参数来指定一个值变换函数和一个反转标志，它们分别默认是 `None` 和 `False`。因为我们通常不用这些选项，可以省略它们以应用默认值。我们还看到过，`dict`、`str.format` 和 `print` 调用也都接受关键字——这些用法我们在前面章节不得不提前引入，因为它们前向依赖于我们终于在本章学到的参数传递模式（唉，那些改变 Python 的人早就懂 Python 了！）。

深度理解

·核心概念：关键字参数是"配置选项"的天然载体——tkinter 组件几十个可配置属性、sorted 的 key/reverse、print 的 sep/end/file——全靠关键字按名指定，无需记忆顺序。

·tkinter 的本质：Button(text=..., command=...) 内部把 options 字典映射为 C 变量与选项；关键字参数让"只设置关心的属性"成为可能——这是第 18.12 节"配置与数据分离"设计哲学的极端体现。

·设计原因：如果只能按位置传配置，sorted(list, None, True) 将不可读、易错；关键字把"语义"带进调用本身。

·作者的感叹："those who change Python already know Python"——学习参数传递不只是学语法，更是解锁阅读/使用标准库的钥匙；此前章节（如第 8 章的 sorted、dict）早已提前使用了这些模式。

·初学者误区：以为关键字参数"性能差"或"只为了可读性"——它是 API 设计的核心契约；也不要吧 key= 误写成 keys=（标准库常见的拼写错误，报 TypeError: unexpected keyword argument）。

脚注 1（关于 mins.py 的排序性能）：实际上，情况很复杂。CPython 的排序（list.sort 和 sorted 共用）用 C 编写，采用高度优化的算法，会尝试利用待排序元素的部分有序性。不过，排序本质上是繁忙的操作（它必须多次切分序列再拼回去），而其他版本只是做一次从左到右的线性扫描。这表明：如果参数部分有序，排序可能更快，否则很可能

更慢。即便如此，Python 的性能变化很频繁；排序用 C 实现本身帮助很大；而且速度差异在很多程序中可能无关紧要。要精确分析，你应该用 `time` 或 `timeit` 模块计时各种方案——第 21 章你会看到怎么做，但如果等不及，示例包里的 `mins-timings.txt` 文件演示了思路。要点：在 CPython 中，对随机参数，非排序版 `min` 更快；对有序参数，则更慢——截至今天如此！

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 参数验证与默认值设计：任何"主参数 + 可选配置"的 API 都应优先采用"位置参数 + 默认值 + 仅关键字配置项"的签名组合（如 `sorted(iterable, , key=None, reverse=False)`），既灵活又防呆。
- 装饰器与包装器：`args`, `kwargs` 的"收集→转发"模式是装饰器、代理、日志、计时、`mock` 的基础设施（第 39 章全面展开）；`functools.wraps` 内部正是这样传播参数的。
- 面向用户的动态调度：GUI 选择、插件系统、测试框架（`unittest.subTest`、`pytest` 的 `fixture` 注入）都依赖"运行时拼参数、调用时解包"。
- 防御性复制与 API 契约：当函数会原地修改传入容器时

，应在文档中写明；调用方需要保留原数据时用 `list.copy()`/切片/`copy.copy()`；纯函数风格用 `tuple` 兜底但注意灵活性损失。

· Monkey-patching 内置：`builtins.print = print3` 式替换（配合日志），以及第 17 章预告的 `open` 自定义，是“模拟器”技术的延伸。

与其他语言（Java/C++）的区别

特性	Python	C++	Java
传递语义	传对象引用 (call by sharing)	按值/按引用/按 const 引用可选	传引用值 (对象引用, 同 Python)
多返回值	返回元组 + 解包赋值	引用参数/输出参数 (&、指针)	返回对象 (如 Pair/record)
可变参数	args 元组、kw args 字典	省略号 ... + valist (类型不安全)	变长参数 T... (数组)
命名参数	关键字参数原生支持	无 (需包装结构体)	无原生支持 (用 builder 模式模拟)
默认值	定义侧 name=value, 调用侧自由覆盖	默认参数 (声明时求值)	重载模拟, 无默认参数语法
仅关键字/仅位	原生与 / 语法	无	无
函数作为参数	一等公民, f(g, args)	函数指针/funcutor	方法引用/函数式接口

· Python 最接近的语言是 Java (同为 call by sharing)；与 C++ 的最大差异是没有独立的“按引用”语法——可变对象原地修改自动承担“输出参数”角色，配合“返回元组”方案，语义反而更简单统一。

· 历史背景：args/kwargs 源自 Python 1.x 的语法；仅关键字参数 (PEP 3102, Python 3.0) 为“任意位置参数 + 配置项”而生；调用侧多星解包 (PEP 448, Python 3.5

) ; 仅位置参数 (PEP 570, Python 3.8) 把内置函数文档中的 / 固化为语言语法。30 多年的功能叠加解释了本章"规则繁琐"的现实。

高级开发者需要掌握的相关知识

- 阅读函数签名: inspect.signature()、defaults、kwdefaults、code.covarnames 等自省工具; 装饰器里用 functools.wraps 保签名。
- 理解在"调用解包、定义收集、扩展解包赋值 (a, b = c 生成列表)"三处的不同规则。
- 性能意识: 默认值对象在定义时创建一次 (time/timeit 验证排序 vs 线性扫描); 大对象传参零拷贝; kwargs 有字典构造开销, 热路径可考虑显式形参。
- 现代类型注解与 TypedDict/dataclass 组合签名设计; functools.partial/partialmethod 对参数的预绑定。

学习建议 (Learning Advice)

- 重要程度: ★★★★★ (5/5) ——参数传递是 Python 函数使用的日常核心, 是阅读任何库代码的必备技能; 仅关键字参数与 args/kwargs 转发模式在面试与工程中几乎必考。
- 应该掌握到什么程度:
 1. 闭卷默写"参数传递三定律", 能用一句话向别人解释 call by sharing;
 2. 能手工推演七步匹配算法在混合模式下的结果 (本测验

6 题全对是及格线) ;

3. 能解释可变默认值陷阱并给出 None 哨兵修复;

4. 能手写 `def tracer(func, pargs, kargs)` 风格的转发器 ;

5. 能读懂 `def f(a, /, b, c=3, ps, e=4, ks)` 并说出每段的含义。

·后续应该学习哪些内容:

·第 19 章: 函数注解 (annotation) ——签名信息的载体, 排序规则不受其影响;

·第 21 章: 陷阱清单 (gotchas) 中的可变默认值专题、计时器案例 (仅关键字参数的实战) ;

·第 39 章: 装饰器——args, kwargs 转发模式的终极形态;

·第 20 章: 生成器与迭代工具, 配合 `func(gen)` 的惰性展开;

·Python 官方文档: PEP 570 (positional-only) 、PEP 3102 (keyword-only) 、Python 参考手册的"调用"一节, 作为排序规则的权威出处。